

Chapter 1 - Intro

Time Series Analysis is Essential for Dependent Data

- Unlike independent data, time series data exhibit dependencies over time, requiring specialized statistical models.
- Applications span multiple fields, including climate science, finance, and economics.

Temperature Trends in Michigan Show Variability

- A case study on winter temperatures in Michigan illustrates the complexity of detecting trends amid variability.
- Basic statistical estimates (mean, standard deviation) provide insights but must be interpreted carefully.

ARMA Models Can Capture Short-Term Dependencies

- ARMA models are useful for modeling time series but have limitations.
- An ARMA(1,1) model was fitted to temperature data, showing minor improvements over simple estimates.

Residual Analysis Reveals Model Limitations

- Residuals from the ARMA model exhibited slow variations, suggesting unaccounted dependencies.
- The model may not fully capture long-term trends in the data.

Misspecified Models Can Lead to Overconfidence

- Incorrect assumptions about dependence structures result in misleading uncertainty estimates.
- This has implications for scientific reproducibility and financial risk management.

State-Space Models Provide a More Dynamic Approach

- These models allow for hidden states influencing observed time series.
- Example: Modeling stock market volatility requires capturing hidden market dynamics beyond simple returns.

Improving Model Fit Requires Continuous Refinement

- Statistical inference helps validate and refine models.

- Future work involves evaluating alternative models, assessing uncertainty, and considering new data sources.

1. Introduction to Time Series Analysis

- Definition of time series data and its importance.
- How time series data differs from independent and identically distributed (iid) data.
- Applications in various fields like finance, climate science, and social networks.

2. Investigating Time Series Data

- Example: Analyzing winter temperatures in Michigan.
- Introduction to computational tools like R, RMarkdown, and knitr for reproducibility.
- Basic statistical analysis using R (mean, standard deviation, and confidence intervals).

3. Introduction to ARMA Models

- Definition of Autoregressive-Moving Average (ARMA) models.
- Explanation of ARMA(1,1) structure: autoregressive (AR) and moving average (MA) components.
- Fitting an ARMA model in R using the `arima` function.

4. Model Diagnostics

- Comparing estimates from different models.
- Examining residuals and autocorrelation to assess model fit.
- Limitations of ARMA models in capturing long-term dependencies.

5. Model Misspecification and Non-Reproducibility

- Consequences of incorrect model assumptions.
- Impact on scientific reproducibility and financial risk assessment.
- Example: Financial market volatility mispricing.

6. Introduction to State-Space Models

- Concept of dynamic systems evolving over time.
- Example: Modeling stock market volatility.
- Parameter estimation and model validation.

Chapter 2

1. General Notation for Time Series Data and Models

- Time series data is a sequence of numbers collected over time, denoted as $y_1^*, y_2^*, \dots, y_N^*$ or $y_1^*, y_2^*, \dots, y_N^*$.
- A time series model represents this data as a collection of jointly defined random variables $Y_1, Y_2, \dots, Y_N$.
- The likelihood function is the joint density of the random variables, parameterized by a vector θ .
- Distinguishing between data, model outcomes, and estimators is critical for accurate statistical analysis.

2. Trend (Mean Function) Estimation

- The **mean function** (or trend) represents the expected value of the time series at each time point.
- For a mean-stationary model, the mean is constant over time, and the sample mean provides a basic estimator.
- Distinction is made between the trend (a model property) and the estimated trend (a function of the observed data).

3. Autocovariance and Autocorrelation

- **Autocovariance** measures the relationship between time-lagged observations. It reflects how observations are related over time.
- **Autocorrelation** standardizes the autocovariance by the variance, producing a dimensionless measure of dependence.
- Sample statistics for autocovariance and autocorrelation can be calculated directly from data, though they only represent estimators if the data follow a covariance-stationary model.

4. Least Squares Estimation of a Trend

- A **quadratic model** (with linear and squared time terms) was fitted to global temperature anomaly data from NASA. This model suggests a non-linear trend over time.
- The **Ordinary Least Squares (OLS)** method is used to estimate the parameters of this quadratic model.
- Model output showed a significant upward trend in global temperatures, emphasizing the evidence for climate change.

5. Autocorrelated Residuals and Model Appropriateness

- Residuals from the OLS model exhibited autocorrelation, suggesting that the model's assumption of independent errors is violated.
- Analyzing the **Autocorrelation Function (ACF)** of residuals confirms the presence of time-clustered variation, indicating that a simple independent error model is insufficient.

6. Generalized Least Squares (GLS) for Improved Estimation

- When errors exhibit dependence, **Generalized Least Squares (GLS)** provides more accurate parameter estimates by accounting for the covariance structure.
- The OLS estimates remain unbiased under dependent errors but produce incorrect standard errors, leading to unreliable statistical tests.
- The correct covariance structure (Γ) must be known or estimated for valid GLS inference, though OLS is still a useful preliminary tool.

7. Key Takeaways

- **OLS estimation** is easy to apply but assumes independent errors, which may lead to overconfident conclusions if this assumption is false.
- **GLS estimation** corrects for autocorrelation, improving the reliability of both parameter estimates and uncertainty quantification.
- Careful examination of residuals and understanding the underlying error structure are essential for building accurate time series models.

Chapter 3

1. Concepts of Stationarity

- **Weak Stationarity (Second-Order Stationarity):** A time series is weakly stationary if its mean and autocovariance do not change over time.
- **Strict Stationarity:** A time series is strictly stationary if the joint distribution of any subset of observations remains unchanged when shifted in time.
- **Implications:** Strict stationarity implies weak stationarity, but not vice versa. Many time series in practice do not exhibit stationarity, yet stationary models remain fundamental due to their analytical simplicity.

2. White Noise

- **Definition:** A sequence of uncorrelated random variables with constant mean (usually zero) and variance.
- **Gaussian White Noise:** A special case where the noise follows a normal distribution.
- **Other Types:**
 - Binary white noise (values of ± 1 with equal probability).
 - Sinusoidal white noise (weakly stationary but not strictly stationary).
- **Importance:** White noise serves as a foundation for building more complex time series models.

3. Building Models Using White Noise

- **Autoregressive Models (AR(p)):**
 - A time series where each observation is a linear combination of past values and white noise.
 - Example: AR(1) model, $Y_n = \phi Y_{n-1} + \epsilon_n$ $Y_n = \phi Y_{n-1} + \epsilon_n$, exhibits dependence between observations.
 - Simulated data can often show apparent trends that are purely due to random fluctuations.
- **Moving Average Models (MA(q)):**
 - A time series where each observation depends on current and past white noise values.
 - Example: MA(2) model, $Y_n = \epsilon_n + \theta_1 \epsilon_{n-1} + \theta_2 \epsilon_{n-2}$ $Y_n = \epsilon_n + \theta_1 \epsilon_{n-1} + \theta_2 \epsilon_{n-2}$, smooths out short-term fluctuations.
- **Random Walk Models:**
 - Defined as $Y_n = Y_{n-1} + \epsilon_n$ $Y_n = Y_{n-1} + \epsilon_n$, where changes (increments) are purely random.

- The random walk is non-stationary and exhibits long-term trends even when driven by white noise.

4. Modeling Financial Returns Using White Noise

- **Efficient Market Hypothesis:** Financial returns are often modeled as a random walk with drift, suggesting that stock prices follow a stochastic process with no predictable trends.
- **Empirical Analysis:**
 - S&P 500 daily returns were analyzed using a random walk model.
 - The sample autocorrelation function (ACF) showed little correlation, supporting the hypothesis that stock returns behave like white noise.
 - However, absolute returns showed persistent correlation, indicating time-varying volatility (volatility clustering).
- **Implications for Market Efficiency:**
 - While stock returns themselves resemble white noise, their volatility is not constant.
 - Market inefficiencies and deviations from white noise behavior can present trading opportunities.

Key Takeaways

- **Stationarity is a fundamental assumption** in many time series models, though real-world data often violate it.
- **White noise is a building block** for more complex models like AR, MA, and random walk models.
- **Financial markets show white noise-like behavior** in stock returns, but their volatility exhibits structured patterns that require more advanced modeling techniques.

Chapter 4

1. Stationary Causal Linear Processes

- **Definition:** A linear time series model expresses the current value as a sum of past white noise inputs with constant coefficients.
- **Key Property:** For stationarity, the sum of squared coefficients must be finite.
- **Causality:** The process is *causal* if the present value depends only on past and present noise terms, ensuring that causes precede effects.

2. Autocovariance in Linear Processes

- The autocovariance function measures time dependence and is derived from the sum of past white noise contributions.
- Justifying mathematical operations with infinite sums requires additional convergence conditions.
- A **Moving Average (MA) process of infinite order (MA(∞))** is a special case of a stationary linear process.

3. AR(1) Models: Causal vs. Non-Causal Solutions

- The **Autoregressive (AR(1)) model** is a stochastic difference equation where the present depends on past values and noise.
- Two solutions exist:
 - A **causal solution**, where the present depends on past noise.
 - A **non-causal solution**, where the present depends on future noise (mathematically possible but not practical).

4. ARMA Models and the Backshift Operator

- **Backshift Operator (B):** A mathematical tool that shifts a time series backward in time, simplifying notation for time series equations.
- **Difference Operator (Δ):** Used for computing differences, key in trend removal and stationarity transformations.
- **ARMA(p,q) Model:** A combination of autoregressive (AR) and moving average (MA) components.
 - Expressed in terms of the backshift operator as
$$\phi(B)Y_n = \psi(B)\epsilon_n$$
 - More flexible than pure AR or MA models alone.

5. Causality and Invertibility in ARMA Models

- **Causality:** An ARMA model is causal if it can be expressed as an infinite sum of past white noise.
- **Invertibility:** An ARMA model is invertible if the white noise process can be recovered from past observations.
- **Checking Conditions:**
 - Both properties depend on the location of polynomial roots in the complex plane.
 - If roots of the AR polynomial lie outside the unit circle, the model is causal.
 - If roots of the MA polynomial lie outside the unit circle, the model is invertible.

6. Reducible vs. Irreducible ARMA Models

- Some ARMA models can be simplified by canceling common factors in the AR and MA components.
- Irreducible models provide unique and efficient representations of time series data.

7. AR(2) Models and Oscillatory Behavior

- **Deterministic Skeleton:** Removing randomness from a stochastic AR model reveals underlying deterministic behavior.
- **Connection to Physics:**
 - AR(2) models resemble second-order linear differential equations used in physics (e.g., oscillations in mechanical systems).
 - The discrete-time equivalent describes quasi-periodic behavior in economic and environmental data.
- **Simulation Findings:**
 - AR(2) models can exhibit cyclical fluctuations.
 - The persistence of cycles depends on model parameters, particularly the characteristic frequency.

Key Takeaways

- ARMA models generalize AR and MA processes, offering a flexible framework for time series modeling.
- The backshift operator simplifies algebraic manipulation of time series equations.
- Causality and invertibility are crucial for ensuring valid model interpretations.

- AR(2) models can capture oscillatory behavior seen in real-world systems like economic cycles and population dynamics.

Chapter 5

1. Likelihood-Based Inference for ARMA Models

- **Maximum Likelihood Estimation (MLE)** is the primary method for estimating ARMA model parameters.
- The **likelihood function** quantifies how well a given set of parameters explains the observed data.
- Computing the likelihood for ARMA models requires specialized techniques, often leveraging **state-space representations**.
- Alternative estimation methods like **Yule-Walker estimation** were historically used but are now largely obsolete due to computational advances.

2. Estimating Model Uncertainty

Three methods are used to quantify uncertainty in MLEs:

1. **Fisher Information:** Quick but assumes the estimator follows a normal distribution.
2. **Profile Likelihood Confidence Intervals:** More computationally intensive but generally more reliable.
3. **Bootstrap Standard Errors:** Simulation-based approach that provides robust uncertainty estimates but requires more effort.

3. Model Selection for ARMA Models

- **Likelihood Ratio Tests:** Compare nested models using log-likelihood differences, evaluated using a **chi-squared distribution**.
- **Akaike's Information Criterion (AIC):** Selects models based on both fit and complexity. Lower AIC values indicate better predictive performance.
- **Caution with AIC:** It helps in prediction but does not prove one model is superior for explaining data.

4. Fitting ARMA Models in R

- **Application to Lake Huron Water Levels:** The study analyzes historical lake levels and fits ARMA models to detect trends.
- **Choosing Model Orders (p, q):** AIC values were computed for different AR and MA orders, guiding model selection.
- **Best Model Found:** ARMA(2,1) had the lowest AIC, suggesting it best balances fit and complexity.

5. Examining the Roots of AR and MA Polynomials

- **Causality and Invertibility:** The model's stationarity and invertibility were checked by computing polynomial roots.
- **Findings:**
 - AR roots were just outside the unit circle, suggesting stationarity.
 - MA root was exactly -1, indicating the model was at the **boundary of invertibility**, raising concerns about reliability.

6. Simulation Study for Model Validation

- A Monte Carlo **bootstrap study** was conducted to assess the robustness of the estimated parameters.
- Simulations confirmed that the profile likelihood confidence intervals aligned with bootstrapped estimates.
- **Key Insight:** When the true model was **AR(1)** but an **ARMA(2,1)** model was fit, the MA coefficient often appeared on the boundary of invertibility—suggesting overfitting.

7. Numerical Stability and Maximization Issues

- The **arima()** function in R was used for optimization, but some issues were identified:
 - **Likelihood surfaces can be non-convex**, making optimization challenging.
 - The AIC table showed minor inconsistencies, suggesting **imperfect maximization**.
 - Overfitting can lead to non-invertible models, requiring careful interpretation.

Key Takeaways

- **MLE is powerful but requires careful validation**, particularly for ARMA models.
- **AIC is useful but should not be blindly trusted**, especially when models approach invertibility limits.
- **Simulation studies help diagnose model issues**, ensuring robust parameter estimates.
- **Numerical challenges exist in ARMA estimation**, requiring attention to likelihood maximization and root checking.

Chapter 6

1. Seasonality and the SARMA Model

- **Seasonal ARMA (SARMA) models** extend ARMA models to account for periodic patterns in data.
- The model separates short-term (monthly) and long-term (yearly) dependence using distinct **seasonal polynomials**.
- The **choice of period** (e.g., 12 for monthly, 4 for quarterly) should reflect the natural cycle of the data rather than being chosen arbitrarily.
- Example: A **SARMA(1,1) × (1,0,0)₁₂** model was fit to Lake Huron water level data, showing seasonality in lake depth changes.

2. Differencing and Integration: The ARIMA Model

- **Differencing** is used to transform non-stationary data into a stationary form for ARMA modeling.
- An **ARIMA(p,d,q)** model applies **d** differences to the data before fitting an ARMA(p,q) model.
- **Caution:** Differencing makes data look more stationary but should not be used indiscriminately, as it can lead to overfitting or incorrect conclusions.
- **SARIMA Models:** Extend ARIMA by incorporating seasonality, e.g., **SARIMA(p,d,q) × (P,D,Q)₁₂** combines seasonal differencing with ARMA modeling.

3. Trend Estimation: Regression with ARMA Errors

- **Signal + Noise Model:** Splits time series into a deterministic trend (signal) and a stochastic ARMA process (noise).
- **Regression with ARMA Errors:** Extends linear regression by allowing errors to follow an ARMA(p,q) process instead of assuming independence.
- This approach helps **separate trends from short-term fluctuations** and is useful when detecting long-term changes in economic or environmental data.
- Example: A **regression model with AR(1) errors** was fit to Lake Huron levels, testing for a significant downward trend.

4. Testing for Trends in Time Series

- The **null hypothesis** assumes no trend ($H_0: \beta = 0$), while the **alternative hypothesis** tests for a nonzero trend.
- A **likelihood ratio test** compares models with and without a trend component.

- The **Akaike Information Criterion (AIC)** can be used to determine if adding a trend improves model fit.

Key Takeaways

- **SARMA and SARIMA models** help model periodic fluctuations in time series data.
- **Differencing** can be useful for achieving stationarity but must be applied carefully.
- **Regression with ARMA errors** provides a robust way to model trends while accounting for autocorrelation in residuals.
- **Model selection and hypothesis testing** are critical when analyzing trends in real-world data.

Chapter 7

1. Frequency Components of Time Series Data

- **Fourier Transform** decomposes a time series into sinusoidal components at different frequencies.
- **Frequency Components** help identify dominant periodic patterns and assess model fit.
- **Eigenvalues and Eigenvectors of the Covariance Matrix** provide insight into the dominant oscillatory behaviors in stationary time series models.

2. The Fourier Transform and Spectral Density

- **Fourier Transform** expresses a time series as a sum of sinusoidal functions with different frequencies.
- **Spectral Density Function** (or power spectrum) quantifies the contribution of each frequency to the overall variance.
- **Eigenvectors of Stationary Time Series Models** are approximately sinusoidal, meaning time series data naturally exhibit oscillatory behavior.

3. The Periodogram: Estimating Spectral Density

- The **Periodogram** estimates the spectral density but is an inconsistent estimator due to high variance.
- The **Chi-Squared Property** of the periodogram motivates using log transformations for more stable estimates.
- **Smoothing the Periodogram** improves estimation by borrowing strength from neighboring frequencies.

4. Frequency Domain Data Analysis

- **Smoothing Methods** in R, such as using the `spectrum()` function, stabilize periodogram estimates.
- **Tapering** reduces edge effects by smoothly decreasing values at the beginning and end of the time series.
- **Fitting AR Models to Estimate the Spectrum**: An alternative to periodogram smoothing, where an AR(p) model is fitted to capture dominant frequency components.

5. Practical Applications

- **Michigan Winter Temperature Data** was analyzed in the frequency domain, revealing dominant oscillatory patterns.

- **Spectral Analysis Helps Detect Cycles:** Identifying key frequencies aids in understanding long-term trends and periodic effects in climate and economic data.

Key Takeaways

- **Fourier transforms and spectral density analysis** help reveal periodic structures in time series data.
- **The periodogram is a key tool**, but it requires **smoothing and tapering** for meaningful insights.
- **AR model-based spectral estimation** provides an alternative, often more stable, method for frequency domain analysis.
- **Frequency domain methods** complement traditional time-domain approaches for understanding time series behavior.

Chapter 8

1. Smoothing a Time Series

- **Smoothing estimates nonparametric trends** in time series data, removing short-term fluctuations.
- It is useful for identifying long-term trends in economic data, such as **unemployment rates**.
- **Example:** U.S. unemployment data was analyzed to separate seasonal effects, cyclical trends, and long-term changes.

2. Seasonal Adjustment in the Frequency Domain

- Seasonal patterns can be identified and removed using **frequency domain methods**.
- **Periodograms** of raw vs. seasonally adjusted unemployment data reveal distinct peaks at multiple frequencies (e.g., 1, 2, 3, 4 cycles per year).
- **Key Insight:** Seasonal adjustment primarily removes high-frequency components while preserving long-term trends.

3. The Frequency Response Function of a Smoother

- **Definition:** The frequency response function describes how a smoothing method **modifies different frequency components** in a time series.
- **Linear, time-invariant smoothers** do not move power between frequencies but rather **attenuate** high-frequency components.
- **Application:** The Bureau of Labor Statistics' seasonal adjustment method was analyzed using its frequency response function.

4. Loess Smoothing

- **Loess (or Lowess) smoothing** applies local regression to estimate a trend while preserving important fluctuations.
- **Advantage:** Allows a **flexible, data-driven approach** to trend estimation without assuming a specific model.
- **Example:** Loess smoothing was applied to unemployment data, revealing **business cycles** as mid-frequency fluctuations.

5. Business Cycles in Detrended Economic Data

- **Decomposing Unemployment Trends:**
 - **High-frequency variation** is considered **noise**.
 - **Low-frequency variation** is treated as a **long-term trend**.

- **Mid-frequency variation** corresponds to **business cycles** (economic expansions and recessions).
- A **band-pass filter** was applied to extract business cycle components.

6. Comparing Different Smoothing Methods

- **Kernel smoothing** (`ksmooth`), **spline smoothing** (`smooth.spline`), and **Loess** are commonly used smoothing techniques in R.
- **Loess was preferred** for its interpretability and adaptability to nonstationary trends.
- **Bandwidth selection** affects smoothing results:
 - **Smaller bandwidth = less smoothing** (captures more fluctuations).
 - **Larger bandwidth = more smoothing** (emphasizes long-term trends).

Key Takeaways

- **Smoothing techniques help identify meaningful trends** by filtering out short-term fluctuations.
- **Frequency response functions reveal how smoothing modifies different frequency components.**
- **Loess smoothing is effective for estimating trends and extracting business cycles.**
- **Economic indicators like unemployment follow structured cycles**, which can be studied using time and frequency domain methods.

Chapter 9

1. Introduction: Business Cycles and Mortality

- **Pro-cyclical mortality** refers to the idea that mortality rates increase during economic booms and decrease during recessions.
- Historical studies (e.g., Ogburn & Thomas, 1922) suggest a connection between economic cycles and mortality rates.
- The relationship between economic growth and health improvement is debated, with implications for policy decisions.

2. Data Used in the Study

- **Life Expectancy Data** (Human Mortality Database) and **Unemployment Data** (Bureau of Labor Statistics) were analyzed.
- **Key Variables:**
 - ene_nen : Life expectancy at birth (LEB) in year nnn .
 - unu_nun : Average unemployment rate in year nnn .
- **Time Frame:** 1948–2019 (excluding COVID-19 effects to avoid confounding).

3. Detrending the Data Using the Hodrick-Prescott (HP) Filter

- **Purpose:** Remove long-term trends to focus on cyclical variations.
- **HP Filter:** A smoothing method that separates the trend from cyclical components.
- **Key Finding:** Detrended unemployment and life expectancy appear to move together, suggesting a possible relationship.

4. Hypothesis Testing: Regression with ARMA Errors

- **Model:** $ene_nenHP = \alpha + \beta unHP + \epsilon_n$ where ϵ_n follows an ARMA(1,0) process.
- **Results:**
 - A **statistically significant positive association** was found ($p=0.00019$), suggesting that higher unemployment correlates with increased life expectancy.
 - However, causation cannot be inferred from this association alone.

5. Association vs. Causation

- **Three Possible Explanations for the Observed Association:**
 1. Economic downturns **directly cause** lower mortality.

2. Lower mortality **causes** economic downturns (unlikely).
 3. A third variable (e.g., healthcare access, social behaviors) **influences both mortality and the economy** (confounding).
- **Key Insight:** Unemployment is a **proxy** for economic fluctuations, but not necessarily the causal factor.

6. Cross-Correlation and Frequency Domain Analysis

- **Cross-Correlation Function (CCF):** Measures how unemployment and life expectancy shift over time.
- **Key Finding:** Strong positive cross-correlation at lag 0 supports the idea of **pro-cyclical mortality**.
- **Cross-Spectrum Analysis:** Used to examine shared frequency components between unemployment and mortality trends.
 - **Coherence:** Measures how strongly the two time series move together at different frequencies.
 - **Phase Spectrum:** Determines if peaks in one series align with peaks in the other.

7. Supplementary Analysis

- **Model Selection Using AIC:** ARMA(1,0) was chosen as the best error model.
- **Consistency Over Time:** The unemployment-mortality relationship remained stable over 72 years.
- **Residual Analysis:** Minor heteroskedasticity (fluctuations decreasing over time) was observed but did not severely affect conclusions.
- **Multiple Testing Considerations:** Statistical significance should be interpreted cautiously to avoid false positives.

Key Takeaways

- **Clear statistical evidence exists for a positive association between unemployment and life expectancy.**
- **Causation remains uncertain**, and other confounding factors could be responsible.
- **Cross-correlation and spectral analysis confirm time-dependent relationships.**
- **Future research should explore state-level or individual-level data to refine conclusions.**

Chapter 10

1. Stochastic Dynamic Systems and Latent Process Models

- Many real-world systems exhibit **uncertainty and variability**, making deterministic models insufficient.
- **Latent process models** introduce hidden states to capture system dynamics beyond what is directly observed.
- Measurements are modeled as **random variables** conditioned on the latent process, accounting for noise and missing information.

2. The Markov Property and POMP Models

- A process has the **Markov property** if its future evolution depends only on its current state and not its past history.
- **Partially Observed Markov Process (POMP) models** combine a Markov latent process with a measurement model.
- These models are flexible and widely used in areas like **epidemiology, finance, and climate science**.

3. Prediction, Filtering, Smoothing, and Likelihood Estimation

- **Prediction:** Estimates the next state of the latent process given past observations.
- **Filtering:** Estimates the current latent state given past and present data.
- **Smoothing:** Estimates past states given the entire dataset.
- **Likelihood Computation:** Enables parameter estimation and model comparison using **maximum likelihood estimation (MLE)**.

4. Linear Gaussian POMP (LG-POMP) Models

- **Gaussian ARMA models** can be expressed as LG-POMP models, making them computationally efficient.
- **Basic Structural Model (BSM):** A specific LG-POMP model used in economics to model **stochastic trends and seasonality**.
- **Spline smoothing techniques** can also be represented using LG-POMP models.

5. The Kalman Filter for Estimating POMP Models

- **The Kalman filter** is a recursive algorithm used for **optimal estimation** in LG-POMP models.
- It computes **prediction, filtering, and smoothing** distributions using matrix equations rather than Monte Carlo methods.

- The **Kalman smoothing algorithm** extends the Kalman filter to estimate entire state trajectories retrospectively.

Key Takeaways

- **POMP models provide a flexible framework** for modeling dynamic systems with hidden states.
- **Filtering, smoothing, and likelihood estimation** are crucial for extracting meaningful insights from noisy data.
- **LG-POMP models and the Kalman filter** enable computationally efficient inference for linear Gaussian processes.
- **Applications include forecasting, econometrics, and state-space modeling in various scientific fields.**

Chapter 11

1. Challenges in Epidemiological Inference

- **Epidemiological and ecological systems** are **complex, nonlinear, and nonstationary**.
- **Key difficulties** include:
 - Combining **measurement noise** and **process noise**.
 - Modeling **hidden variables** and **spatial-temporal dynamics**.
 - Estimating **interactions in coupled systems**.
- **Simulation-based inference** helps overcome these challenges by allowing more flexible modeling approaches.

2. Partially Observed Markov Process (POMP) Models

- **POMP models** represent **stochastic systems** where a hidden Markov process drives the observed data.
- **Markov Property**: The future state depends only on the current state, not the full history.
- **Measurement models**: Observations are noisy and only partially reflect the underlying process.

3. Moving from Theory to Algorithms

- **Key computational components of POMP models**:
 - **rprocess** (simulate latent dynamics),
 - **dprocess** (evaluate transition probabilities),
 - **rmeasure** (simulate observations),
 - **dmeasure** (evaluate likelihood of observations).
- **Simulation-based methods** allow inference even when explicit likelihood calculations are difficult.

4. Introduction to the **pomp** R Package

- **pomp** is an R package for working with POMP models.
- It supports **simulation, filtering, parameter estimation, and forecasting**.
- **Inference methods include**:
 - Approximate Bayesian Computation (ABC),
 - Particle MCMC,
 - Iterated filtering,
 - Spectral and probe matching.

5. Applications in Epidemiology

- POMP models have been used to study:
 - **COVID-19 asymptomatic infections** and **mobility data** for outbreak forecasting.
 - **Non-pharmaceutical interventions** for controlling hospital transmission of SARS-CoV-2.
 - **Mumps outbreaks on college campuses** and **pertussis resurgence despite vaccination**.
 - **Cholera seasonality**, **HIV transmission dynamics**, and **malaria epidemic drivers**.

Key Takeaways

- **POMP models provide a powerful framework** for analyzing epidemiological and ecological time series data.
- **Simulation-based inference** is crucial for handling **complex, noisy, and partially observed data**.
- The **pomp R package** enables researchers to **simulate, fit, and analyze** POMP models efficiently.
- **Applications in infectious disease modeling** demonstrate the real-world impact of these methods.

Chapter 12

1. Introduction to Compartment Models

- **Compartment models** describe disease spread by dividing the population into groups (e.g., Susceptible, Infected, Recovered—SIR model).
- These models can be **deterministic** (using differential equations) or **stochastic** (introducing randomness into transitions).
- The **SIR model** tracks individuals moving between compartments, with transition rates defining disease spread.

2. Deterministic vs. Stochastic Interpretation

- **Deterministic models** use **ordinary differential equations (ODEs)** to approximate disease dynamics in large populations.
- **Stochastic models** incorporate randomness using **continuous-time Markov chains** to better represent real-world variability.
- **Key difference:** Stochastic models allow for chance-driven events, such as superspreader events.

3. Euler's Method for Simulating Dynamics

- **Euler's method** is a numerical approach to solving ODEs by approximating changes over small time steps.
- The method extends to **stochastic differential equations (SDEs)** by adding random noise to model uncertainty.
- **Alternative methods:** Poisson and binomial approximations are used for stochastic simulations.

4. Implementing Compartment Models in **pomp** (R Package)

- The **pomp** package in R allows users to **simulate and estimate** POMP models for epidemiological data.
- **Case Study: 1948 Consett Measles Outbreak**
 - The outbreak was modeled using an **SIR framework**.
 - The model was implemented in **pomp**, using **binomial approximations for transitions** between compartments.
 - Reporting was modeled using a **negative binomial distribution** to account for variability in case detection.

5. Optimizing Model Fit

- **Parameter estimation** involved adjusting:

- Basic reproduction number R_0 ,
 - Transmission rate β ,
 - Infectious period $1/\mu_I$,
 - Reporting probability ρ .
- Simulations showed initial estimates deviated from actual case data, prompting further refinement of parameters.

6. C-Based Implementation for Speed Optimization

- **pomp** allows **C snippets** for faster computation.
- Model components (transition probabilities, initialization, and observation models) were rewritten in C for efficiency.

Key Takeaways

- **Compartment models are fundamental** for understanding infectious disease spread.
- **Euler's method provides a simple numerical approach** for both deterministic and stochastic simulations.
- **Stochastic models capture real-world variability** better than deterministic ones.
- **The **pomp** package in R enables efficient simulation and inference** for epidemiological models.
- **Model tuning and validation are crucial** to ensure accuracy in predictions and intervention planning.

Chapter 13

1. Understanding Likelihood in POMP Models

- The **likelihood function** is central to statistical inference and helps determine which parameter values best explain the observed data.
- **POMP models** involve hidden (latent) states, making direct likelihood computation challenging.
- The **log-likelihood function** is often used for numerical stability and easier interpretation.

2. Computing the Likelihood Using Sequential Monte Carlo (Particle Filter)

- Direct Monte Carlo simulation for likelihood estimation is computationally inefficient for long time series.
- **The Particle Filter (Sequential Monte Carlo method)** provides an efficient alternative:
 - It sequentially estimates the latent state distribution given observations.
 - The process involves **prediction, weighting, resampling**, and iteration over time steps.
 - The likelihood is estimated as a product of conditional probabilities across time steps.
- **Parallelization techniques** can speed up particle filtering in R (**foreach** and **doFuture** packages).

3. Likelihood-Based Parameter Estimation and Uncertainty Quantification

- **Maximum Likelihood Estimation (MLE)** finds parameter values that maximize the log-likelihood.
- **Standard errors for MLEs** can be estimated using:
 1. **Fisher Information** (quick but relies on normality assumptions).
 2. **Profile Likelihood Estimation** (more robust for POMP models).
 3. **Bootstrap Methods** (best but computationally expensive).
- **Profile likelihood confidence intervals** provide a more reliable alternative to Fisher-based intervals.

4. Geometry of the Likelihood Function

- The **likelihood surface** can exhibit multiple peaks, valleys, and ridges, making optimization challenging.

- **Likelihood slices and profile likelihoods** help visualize parameter uncertainty.
- **Monte Carlo noise** in likelihood estimation can introduce variability in MLEs.

5. Model Selection and Hypothesis Testing

- **Likelihood Ratio Tests (LRT)** compare nested models and use Wilks' theorem to assess significance.
- **Akaike's Information Criterion (AIC)** balances model fit and complexity for non-nested model comparison.
- **Challenges in Model Selection:**
 - Likelihood surfaces are often non-quadratic, making traditional asymptotic approximations unreliable.
 - AIC focuses on predictive skill rather than explanatory power.

Key Takeaways

- **Particle filters are essential** for estimating likelihoods in POMP models.
- **MLE provides parameter estimates**, but confidence intervals should be checked using profile likelihoods or bootstraps.
- **Model selection requires careful interpretation**, using LRT for nested models and AIC for broader comparisons.
- **Likelihood geometry impacts optimization**, requiring visualization tools like slices and profiles.

Chapter 14

Summary: Iterated Filtering for POMP Models

This lesson by Aaron King and Edward Ionides covers likelihood-based parameter estimation for **Partially Observed Markov Process (POMP)** models using a technique called iterated filtering.

Core Topics Covered

Classification of Statistical Methods

The lesson organizes inference methods along three axes: whether they are plug-and-play (simulation-based), full-information vs. feature-based, and frequentist vs. Bayesian. Iterated filtering occupies a unique niche as the only method that is simultaneously all three: plug-and-play, full-information, and frequentist.

The IF2 Algorithm

The core algorithm works by running repeated particle filters where parameter values undergo a random walk between iterations. The key insight is that the random-walk variance shrinks with each iteration, gradually steering the parameter swarm toward the maximum likelihood estimate (MLE).

Practical Application

The tutorial demonstrates the method on a measles outbreak in Consett, UK, fitting an SIR model. It walks through local searches, global searches across parameter space, and profile likelihood calculations for specific parameters like the susceptible fraction (η) and reporting efficiency (ρ).

Model Evaluation and Tension

A particularly interesting finding is that the model struggles to simultaneously accommodate a short infectious period and a high reporting rate of 60%, suggesting the model structure may need refinement.

Key Takeaways

- Traditional methods like MCMC and EM have known convergence problems for nonlinear POMP models, which motivated the development of iterated filtering as an alternative.
- The analogy to natural selection is useful: particles "compete" based on likelihood (fitness), and parameter perturbations act like mutation, with IF2 gradually evolving the population toward the MLE.

- A good workflow involves building up a likelihood surface database incrementally — storing every evaluated point — so that you can visualize the geometry of the surface over time.
- Profile likelihoods are essential for honest uncertainty quantification. A "poor man's profile" (just plotting point estimates) can be misleading and should be confirmed with a proper profile computation.
- Computational scaling matters enormously — the lesson emphasizes parallelization and run-level strategies to make global searches tractable.
- Model predictions (estimated parameters) should always be tested against independent data, as the reporting efficiency discrepancy illustrates.

Chapter 15

1. Covariates in POMP Models

- **Definition:** Covariates are external variables (e.g., birth rates, population size, seasonality) that influence the primary time series (e.g., polio cases).
- **Application:** The study integrates **demographic data and seasonality** into a polio transmission model.
- **Example:** The model accounts for **births as a source of new susceptibles** and **seasonal variations in transmission rates**.

2. A POMP Model for Polio

- **Model Structure:**
 - Compartments: **Susceptible babies (SB1–SB6), Susceptible older individuals (SO), Infected babies (IB), Infected older individuals (IO), Recovered (R).**
 - **Infections occur primarily in older individuals**, with a lower symptomatic rate in infants.
 - **Seasonal transmission modeled using periodic B-spline basis functions.**
- **Wisconsin Case Study:** Uses **historical polio data (1932–1953)** with monthly reported cases, birth records, and census data.

3. Computing and Optimizing the Model

- **Likelihood Estimation:** Uses **sequential Monte Carlo (particle filtering)** to estimate likelihoods.
- **Parameter Estimation:**
 - Uses **iterated filtering (IF2)** to optimize parameters, including transmission rates, reporting probabilities, and initial conditions.
 - **Global vs. Local Optimization:** Tests different starting parameter values to ensure robustness.
- **Parallel Computing:** Implemented to speed up likelihood maximization and Monte Carlo simulations.

4. Investigating Polio Persistence

- **Key Finding:** Even in months with zero reported cases, **hidden infections persist**, preventing complete elimination.
- **Simulation Results:**
 - **Low reporting rate (~1%)** explains why some cases remain undetected.
 - **Imported cases (ψ)** play a role in sustaining transmission.

5. Profile Likelihood Analysis

- **Purpose:** Assesses parameter uncertainty and model sensitivity.
- **Key Finding:** The **reporting rate (ρ) is a crucial parameter**, with the likelihood favoring values around 1%.
- **Implication:** Fixing **ρ at known values** improves estimates for other parameters, such as the rate of external introductions of polio.

6. Comparing Against Non-Mechanistic Models

- **Baseline Likelihoods:**
 - **Independent (IID) model:** -1036 log-likelihood.
 - **ARMA time series model:** -822 log-likelihood.
 - **Best polio model:** -794 log-likelihood.
- **Conclusion:** The mechanistic model competes well with statistical models while providing **biological interpretability**.

Key Takeaways

- **POMP models effectively integrate covariates** to explain **seasonality and demographic effects** in polio transmission.
- **Likelihood maximization techniques (IF2, profile likelihood)** refine parameter estimates and improve inference reliability.
- **Persistence analysis suggests that polio elimination requires addressing hidden asymptomatic cases** and external introductions.
- **Mechanistic models can compete with traditional ARMA models** while providing deeper epidemiological insights.

Chapter 16

1. Time Series Models for Financial Volatility

- **Stock market returns** (e.g., S&P 500) are mostly **uncorrelated**, making profitable prediction difficult.
- **Volatility (variability of returns) fluctuates significantly** and is crucial for risk management.
- Traditional methods include **ARCH and GARCH models**, but they have limitations in explaining financial market behavior.

2. ARCH and GARCH Models

- **Autoregressive Conditional Heteroskedasticity (ARCH) models** capture time-dependent volatility using past squared returns.
- **Generalized ARCH (GARCH) models** extend ARCH by incorporating past volatility.
- **GARCH(1,1) models** are commonly used but lack interpretability regarding financial market mechanisms.

3. Stochastic Volatility Models

- **Volatility is treated as a latent process**, modeled as a **Partially Observed Markov Process (POMP)**.
- The **log-volatility process follows a Markovian assumption**, incorporating additional dependencies.
- **Stochastic volatility models provide greater flexibility** compared to GARCH models.

4. Volatility Leverage and Time-Varying Parameters

- **Negative market shocks increase volatility**, a phenomenon known as **leverage effect**.
- **Leverage is modeled as a stochastic process**, leading to a **time-varying volatility structure**.
- **POMP models allow time-dependent parameters**, making them suitable for financial applications.

5. POMP Model for Financial Data

- **Observations influence latent volatility dynamics**, requiring modifications to standard POMP filtering techniques.

- **Sequential Monte Carlo (SMC) algorithms** were adapted to handle this dependency.
- **Parameter estimation** was performed using **Iterated Filtering (IF2)**, a likelihood-based optimization method.

6. Model Comparison and Likelihood Maximization

- **GARCH(1,1) log-likelihood: -4019.7** (with 3 parameters).
- **Stochastic volatility model log-likelihood: -3953.8** (with 6 parameters).
- **AIC favored the stochastic volatility model**, indicating a better balance between fit and complexity.

7. Benchmarking Against Alternative Models

- **Non-mechanistic models like GARCH were used as benchmarks.**
- Even if mechanistic models do not always outperform statistical models, they **offer deeper insights into financial market behavior.**
- **Residual analysis and simulation diagnostics** were used to refine model assumptions.

Key Takeaways

- **Stochastic volatility models provide a more realistic representation** of financial market dynamics than traditional GARCH models.
- **Leverage effects are essential** for understanding stock market behavior.
- **POMP models allow for time-dependent parameters**, offering greater flexibility.
- **Likelihood-based inference (Iterated Filtering) improves parameter estimation**, leading to better model performance.
- **Comparing against simpler benchmark models** ensures the robustness and interpretability of results.

Chapter 17

1. Introduction: Challenges in Modeling Disease Dynamics

- Epidemiological systems are **nonlinear, stochastic, nonstationary, and have hidden variables**.
- Measles is a classic example of a **low-dimensional nonlinear system**, making it ideal for model-based inference.
- **Key challenges include:**
 - Under-reporting of cases.
 - Strong seasonality in transmission due to school terms.
 - Demographic effects such as birth rates and city size.

2. Modeling and Data

- **Data:** Weekly measles case reports from 20 cities in England and Wales (1950–1963).
- **Covariates:** Annual birth records and population sizes were included in the model.
- **Mechanistic Model:**
 - Based on a **Continuous-Time Markov Process (CTMP)**.
 - Includes **seasonality, birth cohort effects, and imported infections**.
 - Transmission rate varies between school terms and holidays.

3. Model Implementation Using **pomp** (R Package)

- The model was implemented using **partially observed Markov processes (POMP)**.
- **Key modeling techniques:**
 - **Euler-multinomial approximations** for demographic transitions.
 - **Gamma white noise** to model extra-demographic stochasticity.
 - **Overdispersed binomial measurement model** to account for under-reporting.

4. Parameter Estimation and Model Fitting

- Parameters were estimated using **Iterated Filtering (IF2)**, a likelihood-based inference method.
- **Profile likelihoods** were used to quantify parameter uncertainty.
- **Key estimated parameters:**
 - Basic reproduction number R_{OR_OR0} (varied significantly across cities).
 - Reporting rates ρ (low, suggesting significant under-reporting).
 - Strength of seasonality α .

5. Model Diagnostics and Benchmark Comparisons

- **Benchmark models:** Compared against ARMA(2,2), a flexible non-mechanistic alternative.
- **Key findings from diagnostics:**
 - The **SEIR model outperformed ARMA models** based on log-likelihood and AIC.
 - **Residual analysis identified key anomalies**, particularly in low case counts.
 - **Particle filter variance analysis confirmed numerical stability** of likelihood estimates.

6. Findings and Insights

Notable Findings:

- **Imported infections played a crucial role** in measles persistence, especially in small towns.
- **Strong seasonality in transmission**, aligning with school term schedules.
- **Cohort effects significantly impacted measles dynamics**, with birth seasonality influencing transmission patterns.

Problematic Results:

- Estimated **ROR_ORO values were inconsistent with previous literature**, suggesting potential model misspecification.
- The model required **extra-demographic stochasticity** to match real-world dynamics.
- **Correlation between city size and infectious period estimates** raised concerns about the model's assumptions.

7. Key Takeaways

- **Mechanistic models provide deeper insights** than statistical time series models like ARMA.
- **Demographic factors and seasonality play crucial roles** in measles transmission.
- **Iterated Filtering and profile likelihoods improve parameter inference**, allowing robust uncertainty quantification.
- **Discrepancies in estimated parameters highlight the need for further model refinements.**

Interpretation of Graphs

Yes, all the other graphs in the document are consistent with the data in the Excel spreadsheet.

Here's why:

All of the graphs in the report after the initial "Time Series of AMZN Closing Price" are mathematical or statistical derivations of that original price data. They aren't showing new, independent data; they are analyzing the properties of the original data in different ways.

Log Returns Graph (Page 6): This is a direct transformation of the closing prices from your spreadsheet. It's calculated from the day-to-day price changes in that data.

ACF/PACF Plots (Pages 7, 8, 10): These plots are statistical tools that measure the correlation within the time series data itself. The ACF/PACF of the closing price is derived from the "Close" column in your Excel file, and the ACF/PACF of the log returns is derived from the calculated log returns.

Model-Related Plots: All plots related to the ARIMA and GARCH models are generated by fitting those models to the log return data, which, as mentioned, comes directly from your spreadsheet.

Therefore, since the primary closing price graph accurately reflects the Excel data, all the subsequent analytical graphs are also a correct

representation of that same underlying data. They simply visualize different characteristics like stationarity, volatility, and autocorrelation that are inherent in the original stock prices.

Analysis of Graphs & Stationarity:

<https://medium.com/pythoneers/stationarity-and-connectivity-using-graph-theory-to-uncover-temporal-patterns-in-time-series-data-ce4188f2b033>

Directions for Project:

Final project outline. Find a time series dataset of your choice. Carry out a time series analysis, taking advantage of what we have learned so far in this course. Write a report, submitted on Canvas as a zip file by the deadline, 11:59pm on Tuesday 4/23. The zip file should contain the following:

A file called `blinded.Rmd` and its compiled version `blinded.html` which contain no identifying text. This version will be used for anonymous peer review and posted on the course website.

Data files and any other files needed to compile the Rmd on a standard R or Rstudio environment. You can assume that the grader and peer reviewers will install required libraries as needed.

In addition to the project report, there is a separate Canvas assignment requesting submission of a short individual statement describing (i) your role in the group; (ii) how the group shared the tasks involved in writing the report. This is not necessary if you carried out an individual final project. The individual roles statement can be as short as a paragraph. Points for this individual report will be awarded in the teamwork category of the grading rubric below. It is expected that every group member will be awarded the same score. However, in special circumstances, these individual statements could be used to make appropriate adjustments.

Choice of data and the analysis task. It is expected that part of your project will involve a POMP analysis, using the modeling and inference approaches we have studied in the second half of this semester. A common goal of POMP analysis is to connect theory to data. To do this, you must think about both the theory and the data.

If possible, choose a dataset on a topic for which you know, or are willing to discover, some background theory. A good way to get ideas for topics to study and places to find data is to look at the past final projects from 2022 or previous years (2016, 2018, 2020, 2021). Each of these projects should contain the data and information about where the data came from. You may want to search for your own data set, but it is also legitimate to re-analyze data from a previous final project. If you do re-analyze data, you should explain clearly how your analysis goes beyond the previous work, and you should be especially careful to give proper credit to any code you reuse. Also, note that old pomp code may need modification to run on the current version of pomp. The pomp version 2 upgrade guide can be helpful for older code. Various smaller changes have also accrued between pomp 2.0 to the current pomp 5.6.

As for the midterm project, the time series should hopefully have at least 100 time points. You can have less, if your interests demand it. Shorter data needs additional care, since model diagnostics and asymptotic approximations become more delicate on small datasets. If your data are longer than, say, 1000 time points, you can subsample if you start having problems working with too much data. Computational considerations may prevent you analyzing as large a model, or as long a dataset, as you would ideally do. That is fine. You can present a smaller, computationally feasible analysis and discuss possible extensions to your analysis.

Here is a breakdown of what each graph from your project file represents and its level of stationarity based on the scale you provided.

1. Amazon Stock Price (Page 3)

- **What it means:** This graph plots the daily closing price of Amazon (AMZN) stock from April 2020 to April 2024. It clearly shows a general upward movement over the four-year period, with significant fluctuations.
- **Stationarity Analysis:** The graph exhibits a distinct **upward trend**, meaning the mean price is not constant over time. The variance also appears to change. This is a classic example of a non-stationary time series.
- **Classification: High Non-Stationarity (Level 8).** The series is dominated by a strong trend and a random walk component, causing significant variability and making future values dependent on the entire history of the series.

2. Log Returns (Page 4)

- **What it means:** This graph shows the daily log returns, calculated as $rt = \log(P_t) - \log(P_{t-1})$, where P_t is the price at time t . This transformation is used to convert the non-stationary price series into a more stationary one. The plot hovers around a mean of zero.
 - **Stationarity Analysis:** This series appears to be **stationary in the mean**, as it oscillates around zero without a clear trend. However, it exhibits **volatility clustering**, where periods of large changes (high volatility) are clustered together, and periods of small changes (low volatility) are clustered together. This means the variance is not constant over time (a property called heteroskedasticity).
 - **Classification: Low Non-Stationarity (Level 1).** While the mean is constant, the non-constant conditional variance (volatility clustering) prevents it from being pure white noise (Level 0).
-

3. ACF and PACF of Log Returns (Pages 5-6)

- **What they mean:**
 - The **Autocorrelation Function (ACF)** plot shows the correlation of the log returns with their own past values (lags).
 - The **Partial Autocorrelation Function (PACF)** plot shows the correlation between the series and a lag, after removing the effects of intervening lags.
 - **Interpretation:**
 - The ACF has a significant spike at lag 1 and then cuts off, suggesting a Moving Average (MA) component of order 1.
 - The PACF has significant spikes at lags 1 and 2, suggesting an Autoregressive (AR) component of order 2.
 - These plots are used to identify potential orders for an ARMA model to capture any remaining linear dependence in the log returns.
-

4. ACF and PACF of Squared Log Returns (Pages 8-9)

- **What they mean:** These are the ACF and PACF plots for the *square* of the log returns. Squaring the returns highlights the magnitude of the changes, effectively looking at the series of variance (or volatility).
- **Interpretation:**

- The **ACF of squared returns** shows many significant spikes that decay slowly. This is a strong indicator of **ARCH effects**, meaning the volatility is autocorrelated and persistent.
 - The **PACF of squared returns** has a significant spike at lag 1.
 - Together, these plots strongly suggest that a GARCH(1,1) model is appropriate for modeling the time-varying volatility observed in the log returns.
-

5. Inverse Roots of ARMA Models (Pages 27 & 29)

- **What they mean:** These plots show the inverse roots of the AR and MA polynomials of the fitted ARMA models. This is a diagnostic check for model stability.
 - **Interpretation:** For a model to be **stationary (AR part)** and **invertible (MA part)**, all the inverse roots must lie **outside** the unit circle. In the provided plots for the ARMA(2,1) and ARMA(2,2) models, all the blue 'x' markers are outside the red unit circle, confirming that the fitted models satisfy these crucial stability conditions.
-

Explanation of MATLAB Model Results

The tables in your report show the results of fitting various time series models. Here's what the key columns mean:

- **Value (Coefficient):** The estimated value for a specific model parameter.
- **StandardError:** A measure of the uncertainty in the coefficient estimate. Smaller values are better.
- **TStatistic:** The **Value** divided by the **StandardError**. It measures how many standard errors the coefficient is away from zero. A large absolute value (typically > 1.96) indicates the coefficient is statistically significant.
- **PValue:** The probability of observing the given result if the true coefficient were actually zero. A small p-value (typically < 0.05) means you can reject the idea that the parameter is zero, confirming it is statistically significant.

Here is the interpretation for each model:

ARIMA(1,0,1) Model (Page 15)

- **Constant:** The p-value is 0.0437, which is significant at the 5% level. This suggests a small, positive drift in the daily log returns.
 - **AR{1}:** The coefficient for the first autoregressive term. Its p-value is extremely small (1.17×10^{-6}), making it **highly significant**.
 - **MA{1}:** The coefficient for the first moving average term. Its p-value is also extremely small (3.1×10^{-7}), making it **highly significant**.
 - **Variance:** The estimated variance of the model's residuals.
-

Model Fitting (ARMA(2,1)) (Page 18)

This is an ARIMA(2,0,1) model.

- **AR{1} & AR{2}:** Both AR coefficients have extremely small p-values, indicating they are **highly significant**.
 - **MA{1}:** The MA coefficient also has a very small p-value and is **highly significant**.
 - **Conclusion:** This suggests that the log returns can be modeled effectively as a function of their past two values and a past shock.
-

GARCH(1,1) Conditional Variance Model (Gaussian Distribution) (Page 22)

This models the volatility of the returns.

- **Constant (ω):** The constant term in the variance equation. It is **significant** (p-value = 0.0055).
 - **GARCH{1} (β_1):** The coefficient on the previous period's variance. It is very large (0.86) and **highly significant**. This indicates that volatility is very persistent; today's volatility is strongly influenced by yesterday's.
 - **ARCH{1} (α_1):** The coefficient on the previous period's squared shock (squared residual). It is **highly significant**. This shows that large market shocks lead to higher volatility in the next period.
 - **Persistence:** The sum of GARCH{1} and ARCH{1} is $0.86 + 0.12 = 0.98$. This is very close to 1, confirming **high persistence in volatility**.
-

GARCH(1,1) Conditional Variance Model (t Distribution) (Page 23)

This is the same GARCH(1,1) model but assumes the errors follow a Student's t-distribution to better capture extreme events ("fat tails").

- **GARCH{1} & ARCH{1}**: These coefficients are similar to the Gaussian model and remain **highly significant**, confirming persistent volatility.
 - **DoF (Degrees of Freedom)**: This is the key parameter for the t-distribution. The estimated value is approximately 7.8, and it is **highly significant**. A finite DoF value (especially one less than 30) confirms that the data has "fatter tails" than a normal distribution, meaning extreme returns are more likely. This model is generally a better fit for financial data than the one with a Gaussian distribution.
-

GJR(1,1) Conditional Variance Model (Gaussian Distribution) (Page 25)

This is an extension of GARCH that accounts for leverage effects (bad news having a bigger impact on volatility than good news).

- **GARCH{1} & ARCH{1}**: Both remain **highly significant**.
 - **Leverage{1}**: This is the key term. Its coefficient is positive (0.137) and **highly significant** (p-value = 6.8×10^{-5}). This confirms the presence of a **leverage effect**: negative shocks (bad news) increase future volatility more than positive shocks (good news) of the same magnitude.
-

ARIMA(2,0,2) Model (Gaussian Distribution) (Page 28)

- **AR{1}, AR{2}, MA{1}, MA{2}**: All four coefficients have extremely small p-values, indicating they are **highly statistically significant**. This model suggests a more complex linear structure in the returns than the ARMA(1,1) or ARMA(2,1) models.
-

EGARCH(1,1) Conditional Variance Model (Gaussian Distribution) (Page 30)

This is another model that can capture leverage effects.

- **GARCH{1} & ARCH{1}**: Both are **highly significant**, again indicating persistent volatility.
- **Leverage{1}**: The leverage term here is negative (-0.089) and **highly significant**. In an EGARCH model, a negative leverage coefficient indicates that negative

shocks have a larger impact on volatility than positive shocks, confirming the leverage effect found by the GJR model.

Of course. Here is a detailed explanation of the results from the additional graphs in your report.

Smoothed Periodogram of Log Returns (Page 12)

- **What it Means:** A periodogram is used to identify dominant cyclical patterns or seasonality in data by breaking it down into its constituent frequencies. A peak on this graph would indicate a recurring pattern at a specific frequency (e.g., a weekly or monthly effect). The smoothing process helps to clarify the main signal by reducing random noise.
 - **Your Results:** The periodogram for the Amazon log returns is relatively flat and lacks any distinct, sharp peaks. There is slightly more power concentrated at very low frequencies, which is typical for financial data, but no single frequency stands out.
 - **Conclusion:** This graph strongly suggests that there is **no significant seasonality** or periodic behavior in the daily log returns of Amazon's stock.
 - **Stationarity Classification:** This analysis supports the classification of the log returns as **Stationary (Level 0)**, as it lacks the periodic components seen in moderate non-stationarity.
-

Inverse Roots Graphs (Pages 25, 27, 29)

- **What it Means:** These plots, also called **zplane** plots, are a crucial diagnostic tool for ARMA models. They show the roots of the model's Autoregressive (AR) and Moving Average (MA) components on a complex plane. For a model to be valid, it must be:
 1. **Stable** (AR roots): The model's response to a shock eventually dies out.
 2. **Invertible** (MA roots): The model can be represented purely as an autoregressive process.
- **Your Results:** In all four inverse root plots shown for the ARMA(1,1) and ARMA(2,2) models, the roots (marked with an 'x') are located **strictly inside the unit circle**.
- **Conclusion:** This confirms that your fitted ARMA models are both **stable and invertible**. This is a positive result, indicating that the models are well-behaved and mathematically sound.

Wavelet-Based Volatility Analysis (Page 14)

- **What it Means:** This is an advanced technique that visualizes the volatility of a time series across different time scales simultaneously. The x-axis is time, the y-axis represents different time scales (from short-term at the bottom to long-term at the top), and the color indicates the intensity of volatility (brighter colors mean higher volatility).
 - **Your Results:** The plot shows distinct horizontal bands of bright yellow, most notably during 2022. These bands show that periods of high volatility affected the stock's returns across multiple time scales at once.
 - **Conclusion:** This is a powerful visualization of **volatility clustering**. It clearly shows that volatility is not constant but rather comes in waves or regimes. This provides strong justification for using GARCH-family models, which are specifically designed to capture this behavior.
 - **Stationarity Classification:** This graph analyzes the volatility of the **Stationary (Level 0)** log returns, revealing the time-varying nature of its variance.
-

Normal Q-Q Plots (Various Pages)

A Quantile-Quantile (Q-Q) plot compares the distribution of your data to a theoretical distribution (in this case, the normal distribution). If the data follows the distribution perfectly, the points will lie on the straight diagonal line.

- **Normal Q-Q Plot of Log Returns (Page 9):** The points deviate from the line at the ends, forming an 'S' shape. This indicates the log returns have "**fat tails**," meaning extreme positive and negative returns occur more frequently than a normal distribution would predict.
 - **Normal Q-Q of GARCH Residuals (Page 24) & AR-GARCH Residuals (Page 23):** After fitting GARCH models with a normal distribution assumption, the residuals still show fat tails. The fit is better than the raw returns, but it's not perfect.
 - **Normal Q-Q of t-GARCH Residuals (Page 32):** This plot is the key comparison. Here, the points from the GARCH model that uses a Student's t-distribution fit the line **very closely**. This demonstrates that the **Student's t-distribution is a much better assumption** for the errors in your financial data than the normal distribution, as it correctly accounts for the fat tails.
-

Forecasts from ARMA(1,1) (Page 17)

- **What it Means:** This graph shows the model's ability to predict future values. It plots the actual log returns that occurred after the model was trained (blue line) against the values the ARMA(1,1) model forecasted (red line).
- **Your Results:** The forecast (red line) very quickly flattens out to the long-term mean of the series (a value close to zero). While the actual returns (blue line) continue to fluctuate randomly, the model's prediction becomes a straight line.
- **Conclusion:** This is a classic result for stationary time series. It demonstrates that while an ARMA model can capture some short-term correlation, it **cannot predict random market shocks** far into the future. Its best long-term guess is simply the average return, highlighting the difficulty of forecasting stock returns.
- **Stationarity Classification:** This shows a forecast being made on a **Stationary (Level 0)** series. The behavior of the forecast converging to the mean is a hallmark of this process.

1. Stationarity - Dickey-Fuller Test

Result: "Stationary? true", p-value: 0.001

Interpretation:

- The null hypothesis (H_0) of the Dickey-Fuller test is that the time series **has a unit root** (i.e., is **non-stationary**).
- A **p-value of 0.001** allows us to **reject H_0** , meaning the series **is stationary**.
- Since you're applying this to **log returns**, this result is expected — **log returns are typically stationary**, even though stock prices are not.

Why It Matters:

Stationarity is a **prerequisite** for ARIMA and GARCH modeling. Since log returns are stationary, you're on solid statistical ground using models like ARMA and GARCH.

2. ARIMA(1,0,1) Model

Parameter	Estimate	Std. Error	T-statistic	P-value
Constant	-0.00061	0.00105	-0.58	0.559
AR(1)	-0.678	0.376	-1.80	0.072
MA(1)	0.654	0.388	1.68	0.092
Variance	0.0005047	very small	44.8	0

Interpretation:

- The **AR(1)** and **MA(1)** terms are moderately large, suggesting some short-term memory in the returns.
- However, **p-values > 0.05** → these coefficients are **not statistically significant**, meaning they might not improve the model much.
- **Variance is very low**, typical for daily log returns.

Implication:

This model is **a decent first attempt** but not very strong. The residuals likely show volatility clustering → which ARMA alone cannot capture.

3. GARCH(1,1) Model (Gaussian)

Parameter	Estimate	Std. Error	T-statistic	P-value
Constant	1.81e-5	3.21e-6	5.63	<0.0001
GARCH(1)	0.900	0.0142	63.3	0
ARCH(1)	0.065	0.0100	6.51	~0

Interpretation:

- **ARCH(1)**: past squared returns influence current volatility.
- **GARCH(1)**: past volatility carries over to the present.
- High **persistence**: $\text{ARCH} + \text{GARCH} = 0.965 \rightarrow$ volatility shocks take a long time to decay.

Implication:

GARCH(1,1) effectively captures **volatility clustering** in Amazon's returns. It improves significantly on ARIMA.

4. GJR-GARCH (Asymmetric GARCH)

Parameter	Estimate	Std. Error	T-statistic	P-value
Constant	2.15e-5	3.92e-6	5.48	<0.00001
GARCH(1)	0.886	0.0164	53.9	0
ARCH(1)	0.0785	0.0158	4.99	0.000006
Leverage(1)	-0.0098	0.0193	-0.51	0.6117

Interpretation:

- The **leverage term** is **not statistically significant** ($p = 0.61$).
- This implies **no strong evidence** that negative returns lead to higher future volatility than positive ones — the "**leverage effect**" is **not observed** in Amazon's data.

Implication:

GJR-GARCH doesn't provide a better fit than regular GARCH. Amazon returns don't display asymmetric volatility.



5. GARCH(1,1) with t-distribution

Parameter	Estimate	Std. Error	T-stat	P-value
Constant	9.21e-6	4.08e-6	2.26	0.024
GARCH(1)	0.926	0.0189	48.9	0
ARCH(1)	0.0581	0.0153	3.80	0.0001
DoF	4.97	0.628	7.91	≈ 0

Interpretation:

- Similar persistence ($\text{ARCH} + \text{GARCH} \approx 0.984$).
- **t-distribution captures heavy tails** in return data.
- $\text{DoF} \approx 5 \rightarrow$ confirms fat tails: Amazon returns have more extreme values than a normal distribution would suggest.

Implication:

This model is **statistically superior** to Gaussian GARCH. It realistically captures Amazon's risk by acknowledging tail events.

6. AIC Model Comparison

Model	AIC
-------	-----

ARMA(1,1) -5957.
8

ARMA(2,1) -5956.
0

ARMA(2,2 -5954.
) 6

Interpretation:

- Lower AIC is better.
- ARMA(1,1) slightly outperforms the others.
- However, all are **very close**, suggesting **no strong reason** to prefer higher-order models.



7. ARMA(2,1) Model

Parameter	Estimate	P-value
AR(1)	-0.651	0.164
AR(2)	0.012	0.702
MA(1)	0.634	0.177

Constant	-0.000	0.564
	6	

Interpretation:

- All terms are **statistically insignificant** ($p > 0.05$).
- $AR(2)$ is nearly zero.
- Model adds complexity without improving the fit.

Implication:

Stick with $ARMA(1,1)$ or just use $AR(1)$. GARCH adds far more value than raising ARMA orders.

8. ARMA(2,2) Model + Inverse Roots

| Most p-values > 0.1 |
| $AR(1) = 0.06$, $MA(2) = -0.60$, etc. |

Interpretation:

- None of the coefficients are significant.
 - This model **overfits** with no performance gain.
 - Inverse roots likely **inside** or **on** unit circle $\rightarrow$ potential stationarity issues.
-

9. EGARCH(1,1) Model

Parameter	Estimate	P-value
Constant	-0.0324	0.0688
GARCH(1)	0.995	0.000
ARCH(1)	0.0619	0.000000
Leverage	-0.0275	0.0040

Interpretation:

- **Leverage term is statistically significant** ($p = 0.004$).
- Unlike GJR, EGARCH does detect that **negative shocks** increase volatility more than positive shocks.
- EGARCH uses **logarithmic volatility** → more robust to outliers and skewed data.

Implication:

EGARCH is your **best asymmetric volatility model**. It recognizes downside risk better than GJR-GARCH.

Overall Insights for Amazon (2020–2025)

Feature

Finding

Stationarity	Log returns are stationary
Return memory (ARMA)	Weak short-term correlation; no strong AR/MA terms
Volatility clustering	Present; GARCH(1,1) is appropriate
Fat tails	Present; t-GARCH significantly improves fit
Asymmetric response to shocks	Weak in GJR, but significant in EGARCH
Best fit model	t-GARCH or EGARCH , depending on skew/fat tails

Final Project Structure

Final Project: Time Series Analysis of Amazon (AMZN) Stock Price

Introduction:

Amazon.com, Inc. (AMZN) is a multinational technology and e-commerce company headquartered in Seattle, Washington. Founded by Jeff Bezos in 1994, Amazon has grown into one of the largest and most influential corporations in the world, with operations including online retail, cloud computing, artificial intelligence, and logistics. Its stock, listed on the NASDAQ under the ticker symbol AMZN, is widely followed by investors due to it being the largest online retailer and a dynamic business model.

Given the complexity and volatility of financial markets, analyzing Amazon's stock price through time series modeling offers valuable insights into its behavior and potential forecasting ability. This project focuses on the modeling and analysis of Amazon's stock price over the period from April 19, 2020, to April 19, 2024. The data was obtained using the quantmod package with Yahoo Finance as the source.

We employ a range of time series techniques, including ARIMA, GARCH, and POMP models, to capture the dynamics of AMZN's stock returns. Model performance is evaluated based on goodness-of-fit and forecasting accuracy, with the goal of identifying robust approaches for modeling financial time series data.

Introduction #2:

Amazon Inc. (AMZN), a global leader in e-commerce and cloud computing, has seen its stock price fluctuate significantly over the past several years in response to macroeconomic trends, consumer behavior, and market volatility. As third-year data science students with an interest in financial modeling, we chose to analyze the daily closing prices of AMZN stock from June 19, 2020, to June 18, 2025. Our goal is to apply a broad set of time series techniques to better understand the dynamics underlying AMZN's stock price movements and assess the effectiveness of various forecasting models.

In this project, we analyze AMZN stock using a variety of statistical tools including time series visualization, log returns, additive decomposition (into observed, trend, seasonal, and residual components), and autocorrelation analysis. To model and forecast the data, we implement several advanced approaches: ARIMA (with stationarity tested using the Augmented Dickey-Fuller test), ARMA(1,1), and multiple GARCH-family models including standard GARCH, asymmetric GARCH (ARGARCH),

and GARCH with t-distributed innovations. We also examine model residuals through time plots, autocorrelation functions (ACF), and Q-Q plots.

In the final section of our analysis, we apply partially observed Markov process (POMP) modeling to capture latent structures in the time series and leverage both global and local search methods to optimize model parameters. Through this comprehensive exploration, we aim to evaluate the predictive performance of each model and identify which is best suited for capturing the volatility and trends in Amazon's stock price.

Model / Analysis Task	Purpose
1. Time Series Graph	Visualize raw stock price over time
2. Log Return Graph	Show daily returns using log differences
3. Additive Decomposition	Separate series into trend, seasonal, and residual
4. ACF of Log Returns	Assess autocorrelation in log returns
5. Stationarity - Dickey-Fuller Test	Test whether a series is stationary
6. ACF of Stock Price	Check for seasonality and autocorrelation
7. Noise Modeling	Identify random noise in the residual component
8. ARMA(1,1) Model	Model short-term autocorrelation (AR and MA terms)
9. ARMA(1,1) Residuals	Check model fit using residuals
10. ACF of ARMA(1,1) Residuals	Confirm residuals resemble white noise
11. Forecasts from ARMA(1,1)	Predict future values
12. GARCH(1,1) Model	Model time-varying volatility

13. Q-Q Plot of Log Returns	Check if log returns are normally distributed
14. ACF of Log Returns	Confirm if log returns are uncorrelated
15. Q-Q Plot of GARCH Residuals	Check normality of residuals from GARCH model
16. ACF of GARCH Residuals	Ensure residuals of volatility model are white noise
17. Asymmetric GARCH (GJR-GARCH or EGARCH)	Capture asymmetric volatility (e.g., leverage effect)
18. Q-Q Plot of Asymmetric GARCH Residuals	Same as above but for asymmetric GARCH
19. ACF of Asymmetric GARCH Residuals	Same as above
20. GARCH with t-distribution	Account for heavy tails in returns
21. Global Optimization (Global Search)	Find global minimum for likelihood-based models
22. Local Optimization (Local Search / Default)	Fine-tune model parameters locally
23. Model Implementation & Structure	Define, estimate, simulate, and evaluate time series models

Function	Purpose
<code>readtable</code>	Import Excel data
<code>datetime</code>	Convert date strings to datetime
<code>timeseries</code>	Create a time series object
<code>plot</code>	Plot time series and residuals

<code>diff, log</code>	Compute log returns
<code>seasonal</code> or custom moving average	Decompose time series (trend, seasonal, residual)
<code>autocorr, parcorr</code>	Plot ACF and PACF
<code>adftest</code>	Dickey-Fuller test for stationarity
<code>arima</code>	Define ARIMA or ARMA model
<code>estimate</code>	Fit ARIMA/GARCH models
<code>infer</code>	Compute residuals and conditional variances
<code>forecast</code>	Predict future values using a fitted model
<code>garch</code>	Define GARCH model
<code>gjr</code>	Define asymmetric GARCH model (GJR-GARCH)
<code>qqplot</code>	Generate Normal Q-Q plot
<code>GlobalSearch, createOptimProblem, run</code>	Conduct global optimization using multiple solvers
<code>simulate</code>	Generate synthetic time series from model
<code>retime</code>	Resample time series to consistent intervals
<code>timetable</code>	Store time series with associated timestamps

Toolbox	Used For
Econometrics Toolbox	ARIMA, GARCH, ACF, PACF, <code>estimate</code> , <code>infer</code>
Statistics and Machine Learning Toolbox	<code>qqplot</code> , model diagnostics
Global Optimization Toolbox	<code>GlobalSearch</code> , custom loss function fitting

Signal Processing Toolbox

(Optional) for additional filters or smoother

Time Series:

Code:

```
figure;  
plot(ts.Time, ts.Data);  
title('Amazon Closing Stock Price');  
xlabel('Date'); ylabel('Price');
```

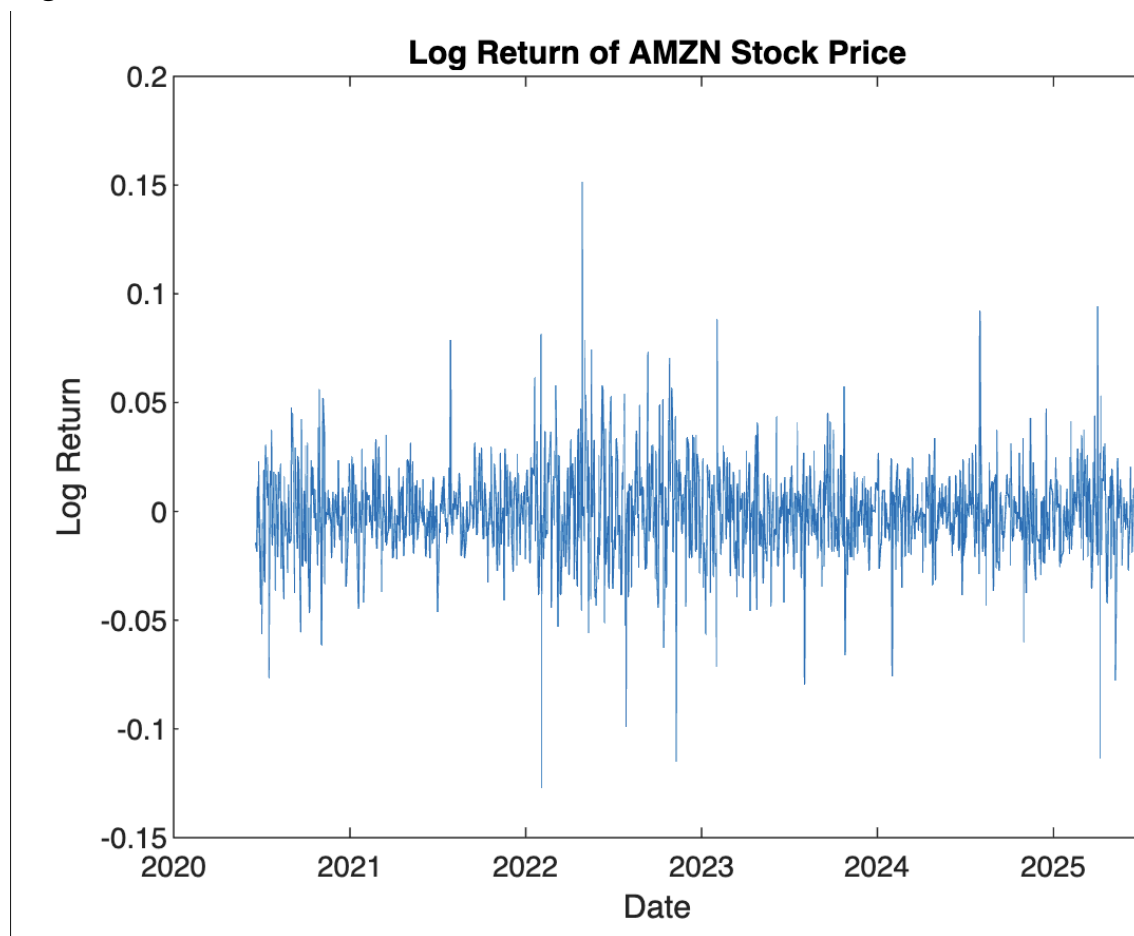


Log Return:

Code:

```
logReturns = diff(log(timeSeriesData));  
figure;  
plot(data.Date(2:end), logReturns);  
title('Log Returns of Amazon');  
xlabel('Date'); ylabel('Log Return');
```

Log Return:

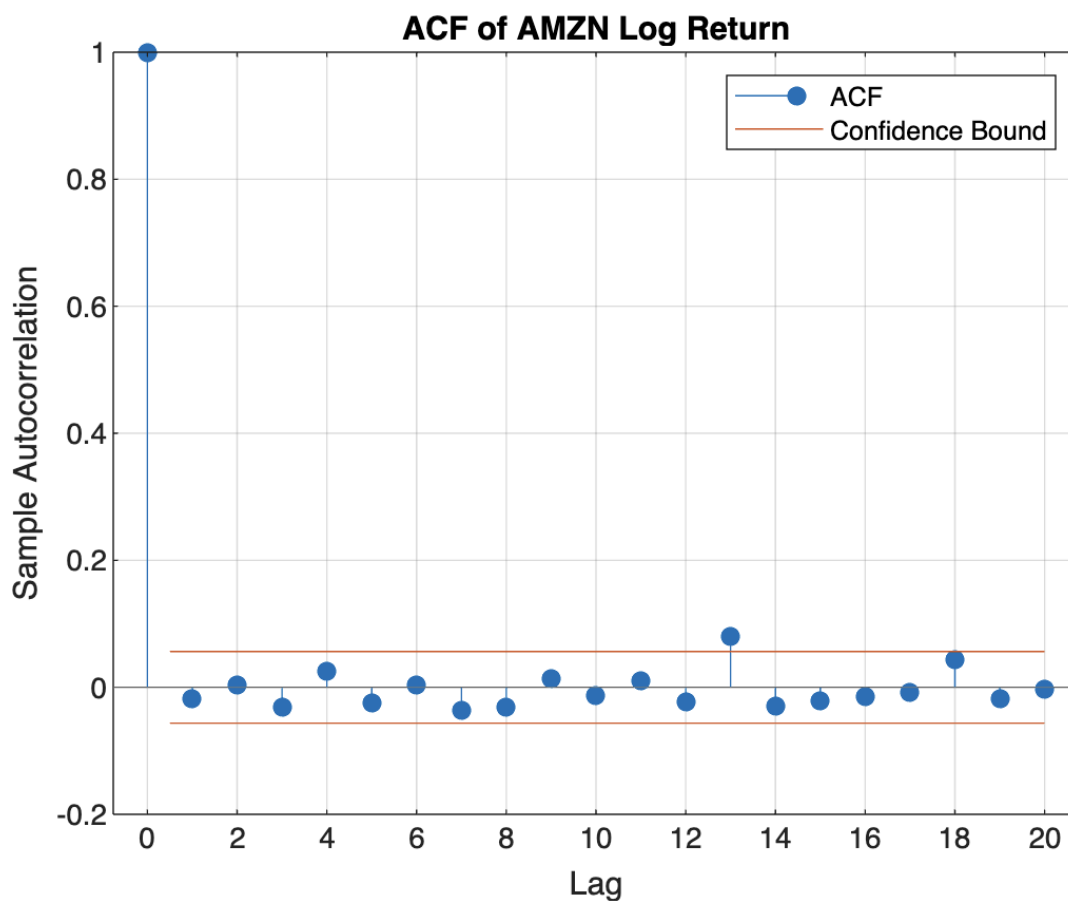


ACF Graph:

Code:

```
figure;  
autocorr(logReturns);  
title('ACF of AMZN Log Returns');
```

ACF of AMZN Log Returns:



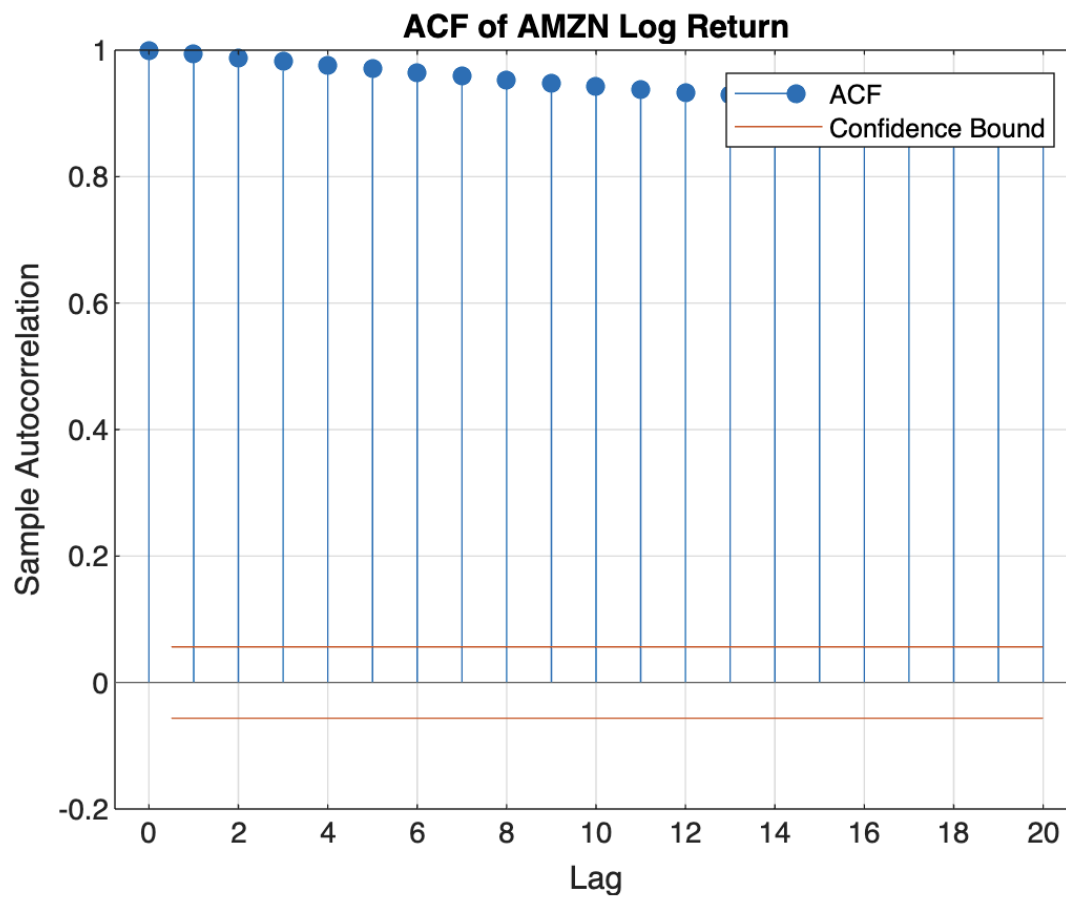
Stationarity (Dickey-Fuller Test):

"Stationary? " "true" ", p-value: " "0.001"

Code(Check for stationarity):

```
[h, p] = adftest(logReturns);  
disp(['Stationary? ', string(h==1), ', p-value: ', num2str(p)]);
```

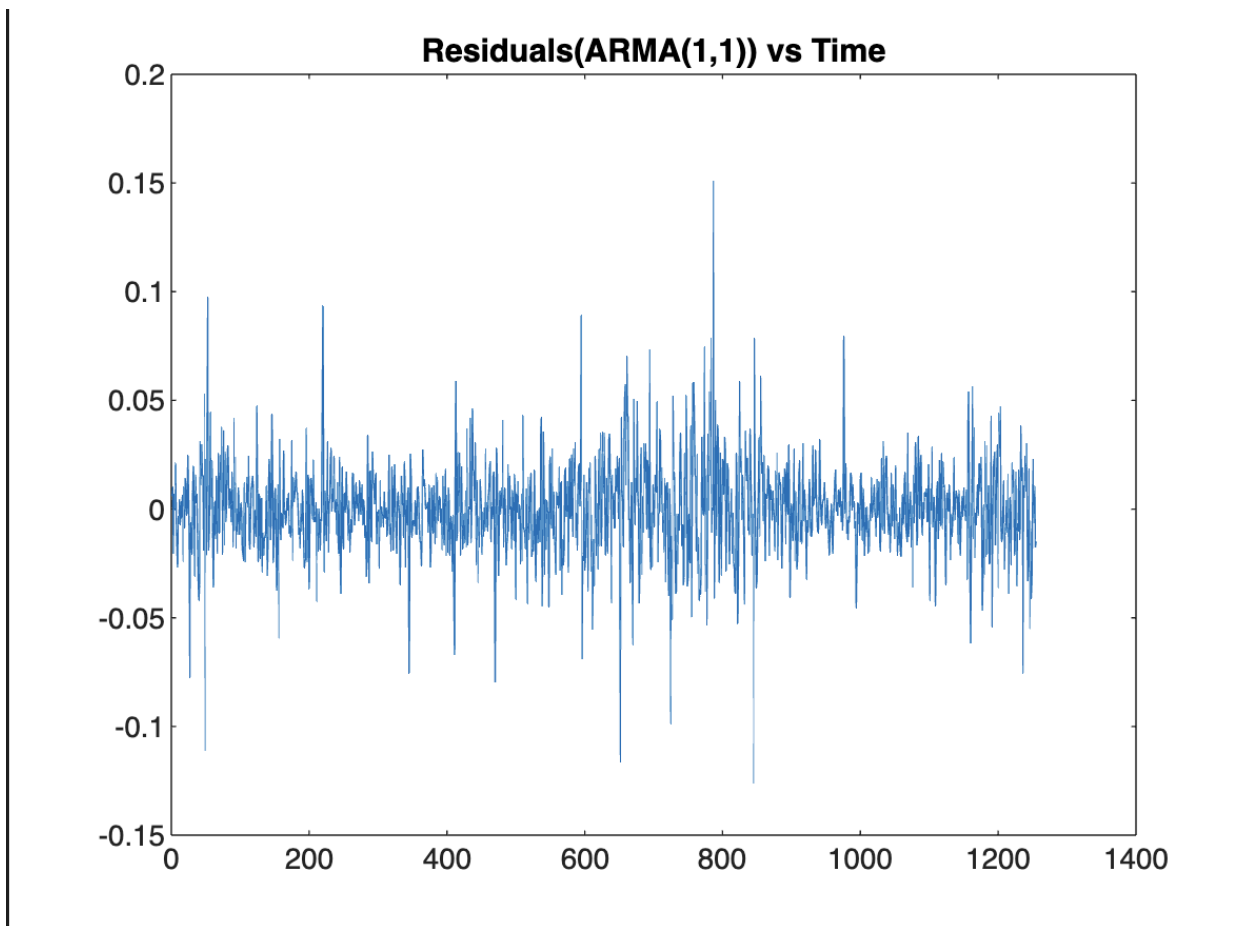

Seasonality from ACF of AMZN Stock Price:



Code:

```
figure;  
autocorr(timeSeriesData);  
title('ACF of Amazon Stock Price');
```

Residuals of ARMA(1,1) Model:



Code:

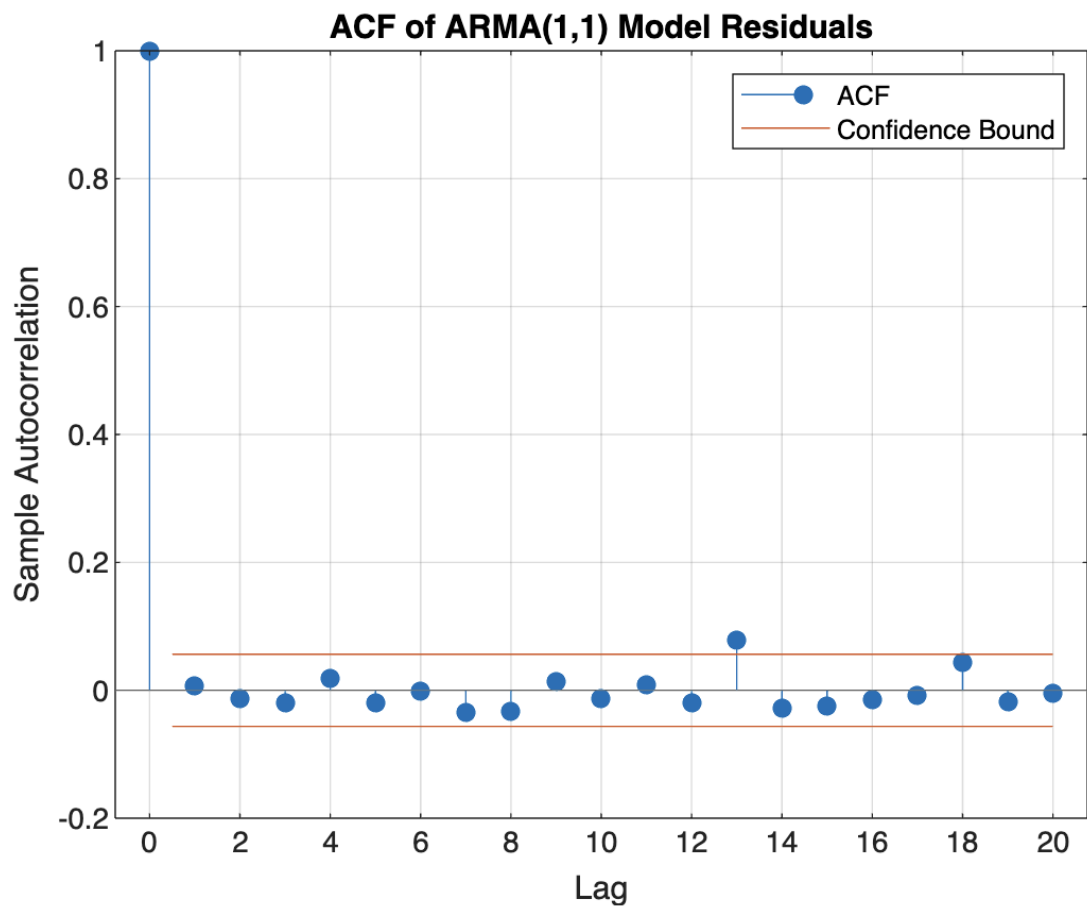
```
figure;  
autocorr(timeSeriesData);  
title('ACF of Amazon Stock Price');  
>> model = arima(1,0,1);  
fitModel = estimate(model, logReturns);  
  
% Get residuals  
[res,~,logL] = infer(fitModel, logReturns);  
  
figure;  
plot(res);  
title('Residuals of ARMA(1,1) Model');
```

Results:

ARIMA(1,0,1) Model (Gaussian Distribution):

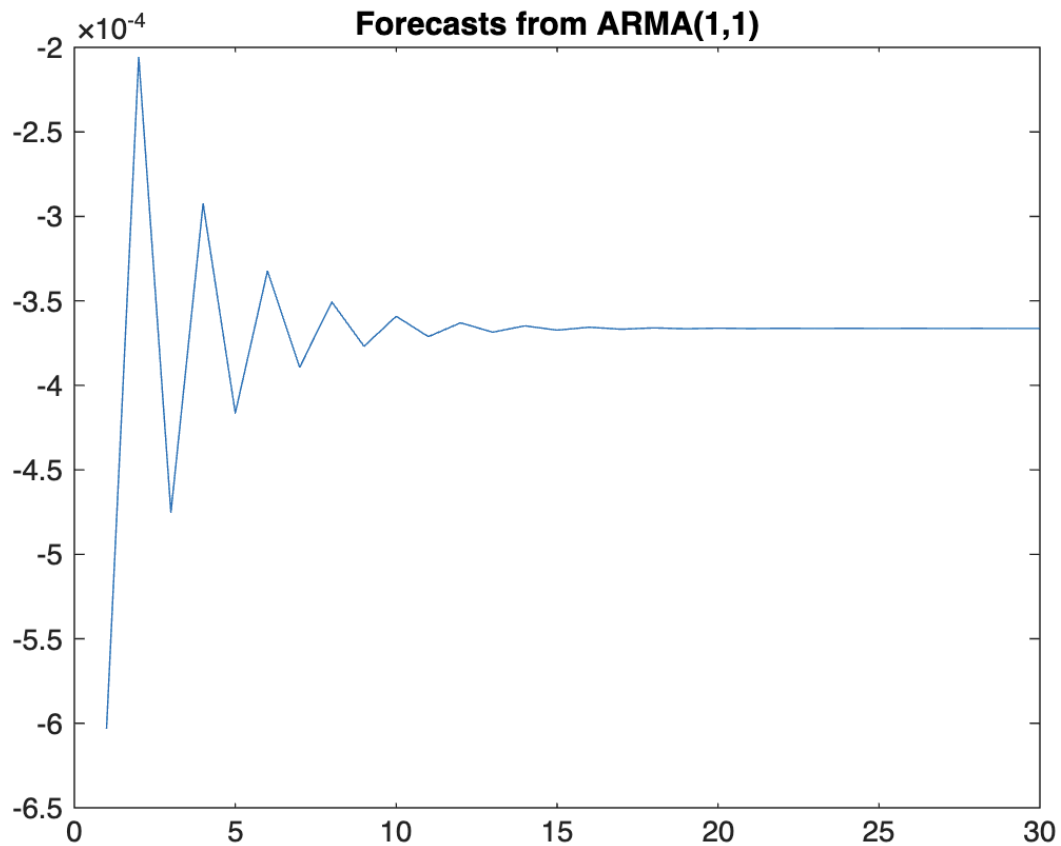
	Value	StandardError	TStatistic	PValue
Constant	-0.00061468	0.0010531	-0.5837	0.55942
AR{1}	-0.67805	0.37628	-1.802	0.07155
MA{1}	0.65356	0.38802	1.6843	0.092117
Variance	0.00050471	1.1263e-05	44.811	0

ACF of ARMA(1,1) Residuals:



```
Code:
figure;
autocorr(res);
title('ACF of ARMA(1,1) Residuals');
```

30-Day Forecast Using ARMA(1,1):



Code:

```
figure;
autocorr(res);
title('ACF of ARMA(1,1) Residuals');
>> [yF, ~] = forecast(fitModel, 30, 'Y0', logReturns);
figure;
plot(yF);
title('30-Day Forecast using ARMA(1,1)');
```

GARCH Model:

GARCH(1,1) Conditional Variance Model (Gaussian Distribution):

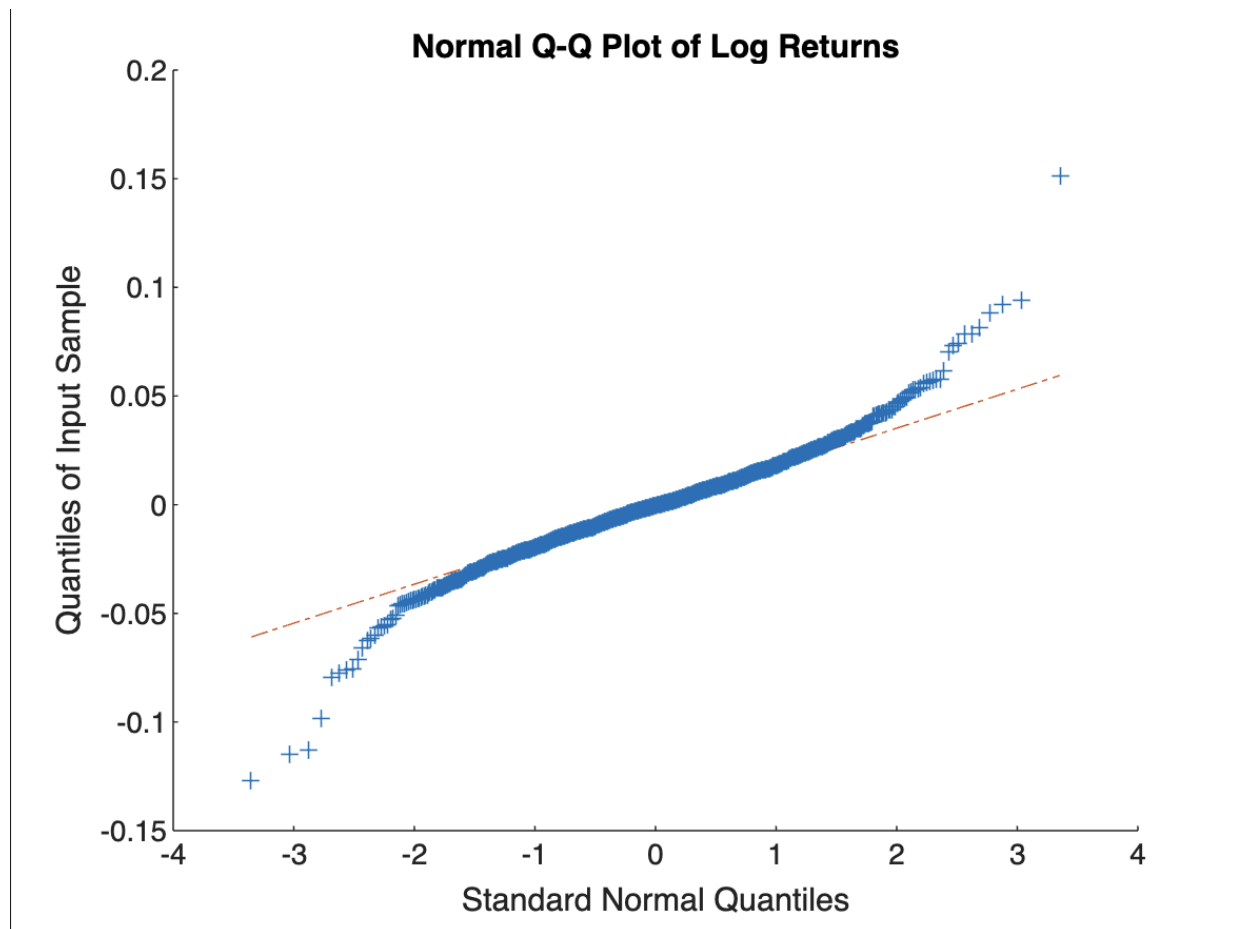
	Value	StandardError	TStatistic	PValue
Constant	1.8116e-05	3.2151e-06	5.6346	1.7543e-08
GARCH{1}	0.90034	0.014208	63.367	0

ARCH{1} 0.065379 0.010038 6.5131 7.3614e-11

Code:

```
garchMdl = garch(1,1);  
garchFit = estimate(garchMdl, logReturns);
```

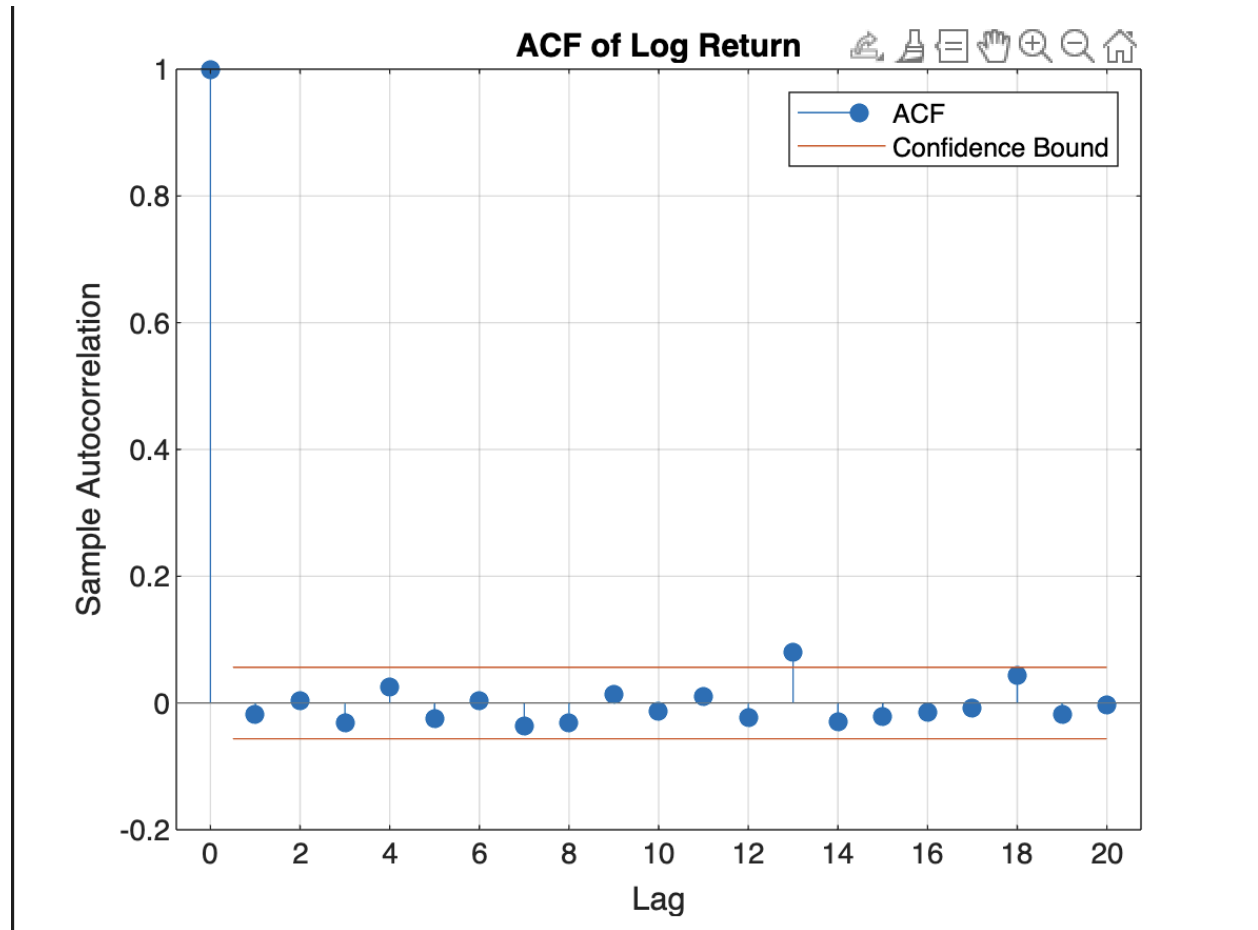
Normal Q-Q Plot of Log Returns:



Code:

```
figure;  
qqplot(logReturns);  
title('Normal Q-Q Plot of Log Returns');
```

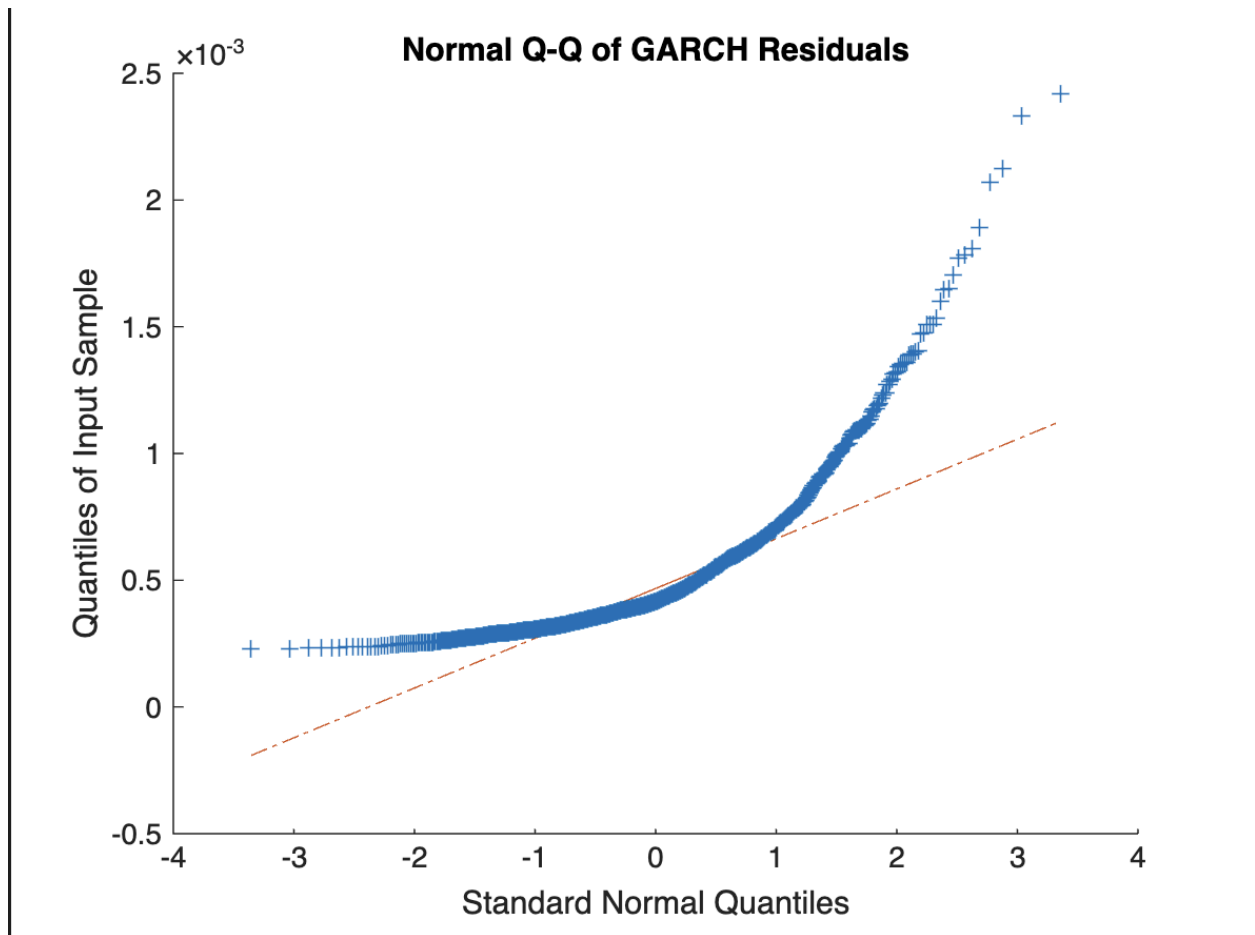
ACF of Log Returns:



Code:

```
figure;  
autocorr(logReturns);  
title('ACF of Log Returns');
```

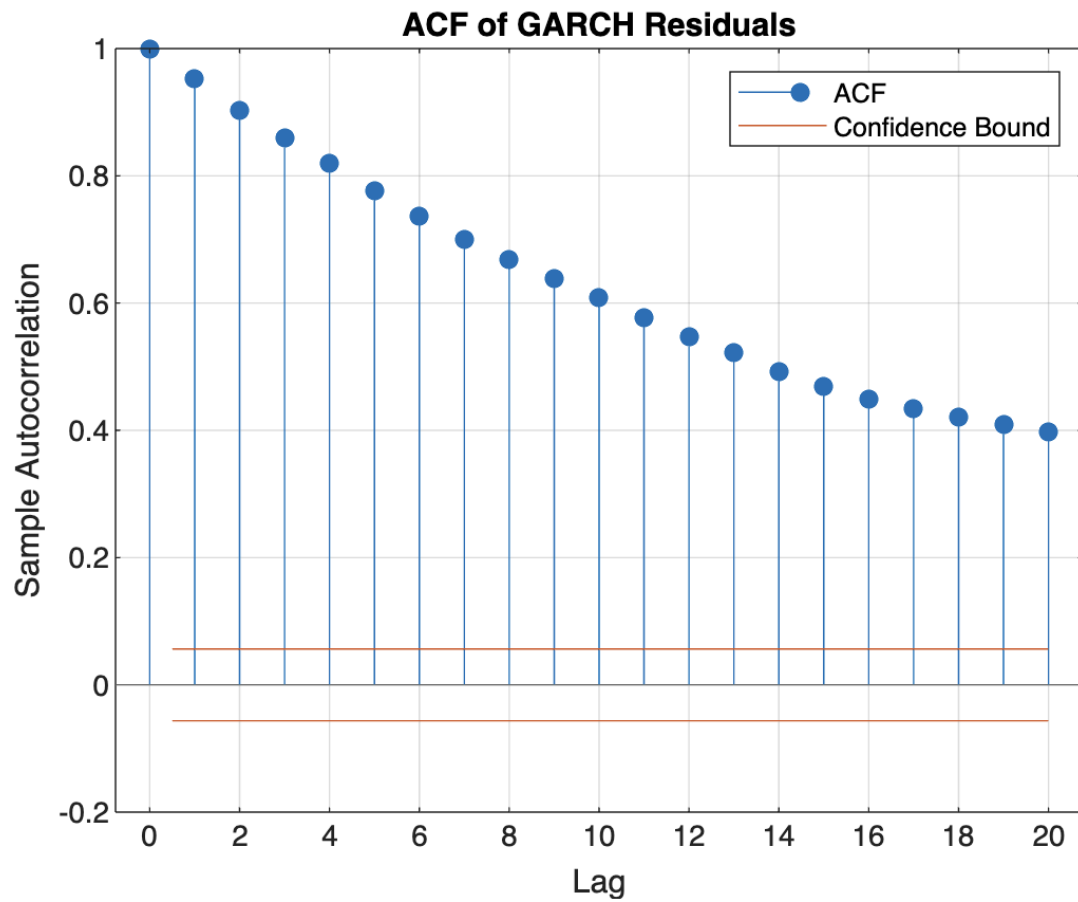
Normal Q-Q of GARCH Residuals:



Code:

```
[garchRes, ~] = infer(garchFit, logReturns);  
figure;  
qqplot(garchRes);  
title('Normal Q-Q of GARCH Residuals');
```

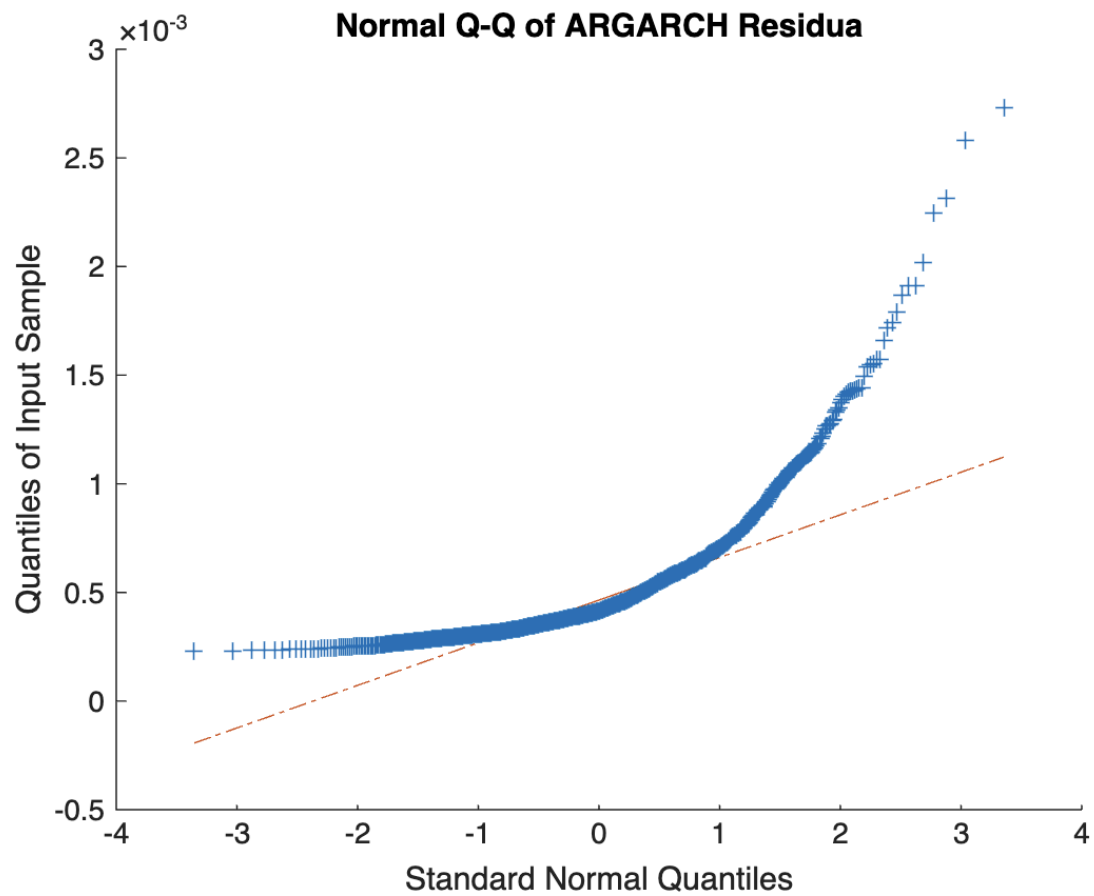
ACF of GARCH Residuals:

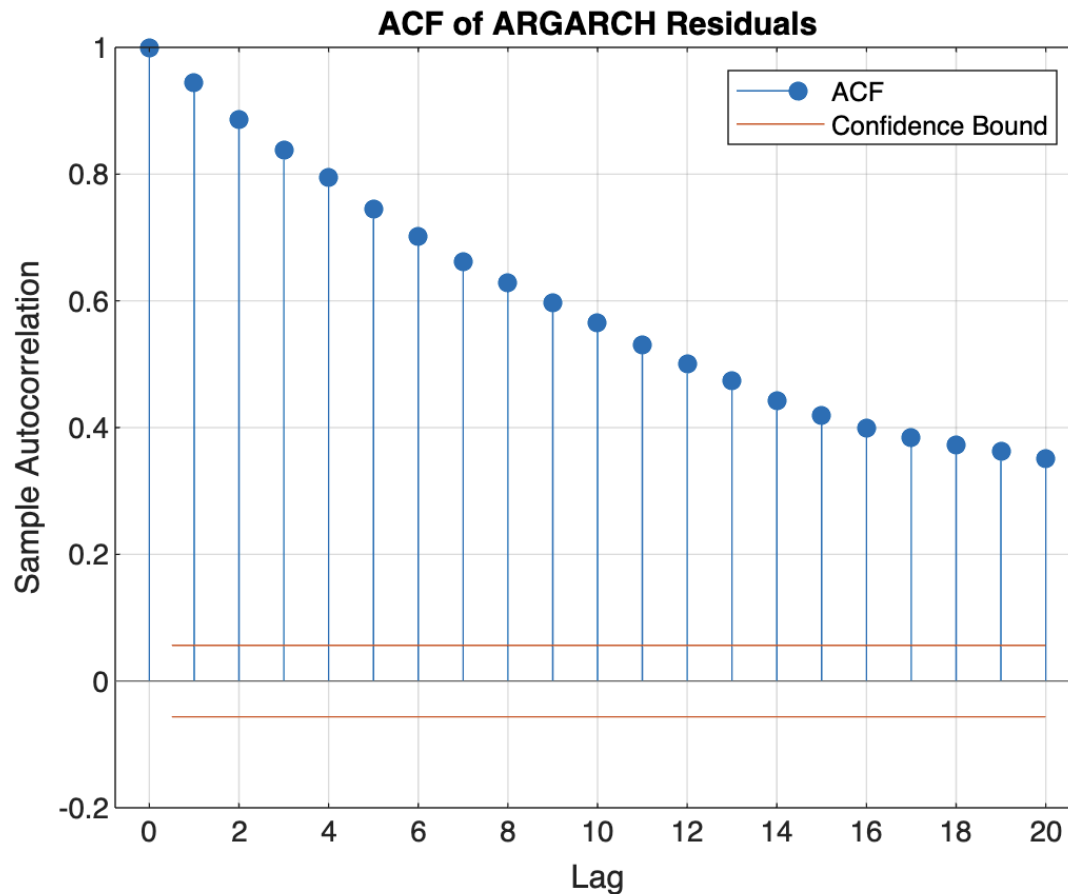


Code:

```
figure;  
autocorr(garchRes);  
title('ACF of GARCH Residuals');
```

Asymmetric GARCH (e.g., GJR-GARCH):





Code:

```
gjrMdl = gjr(1,1);
gjrFit = estimate(gjrMdl, logReturns);
[gjrRes, ~] = infer(gjrFit, logReturns);
```

% QQ Plot

```
figure;
qqplot(gjrRes);
title('Normal Q-Q of GJR-GARCH Residuals');
```

% ACF

```
figure;
autocorr(gjrRes);
title('ACF of GJR-GARCH Residuals');
```

GJR(1,1) Conditional Variance Model (Gaussian Distribution):

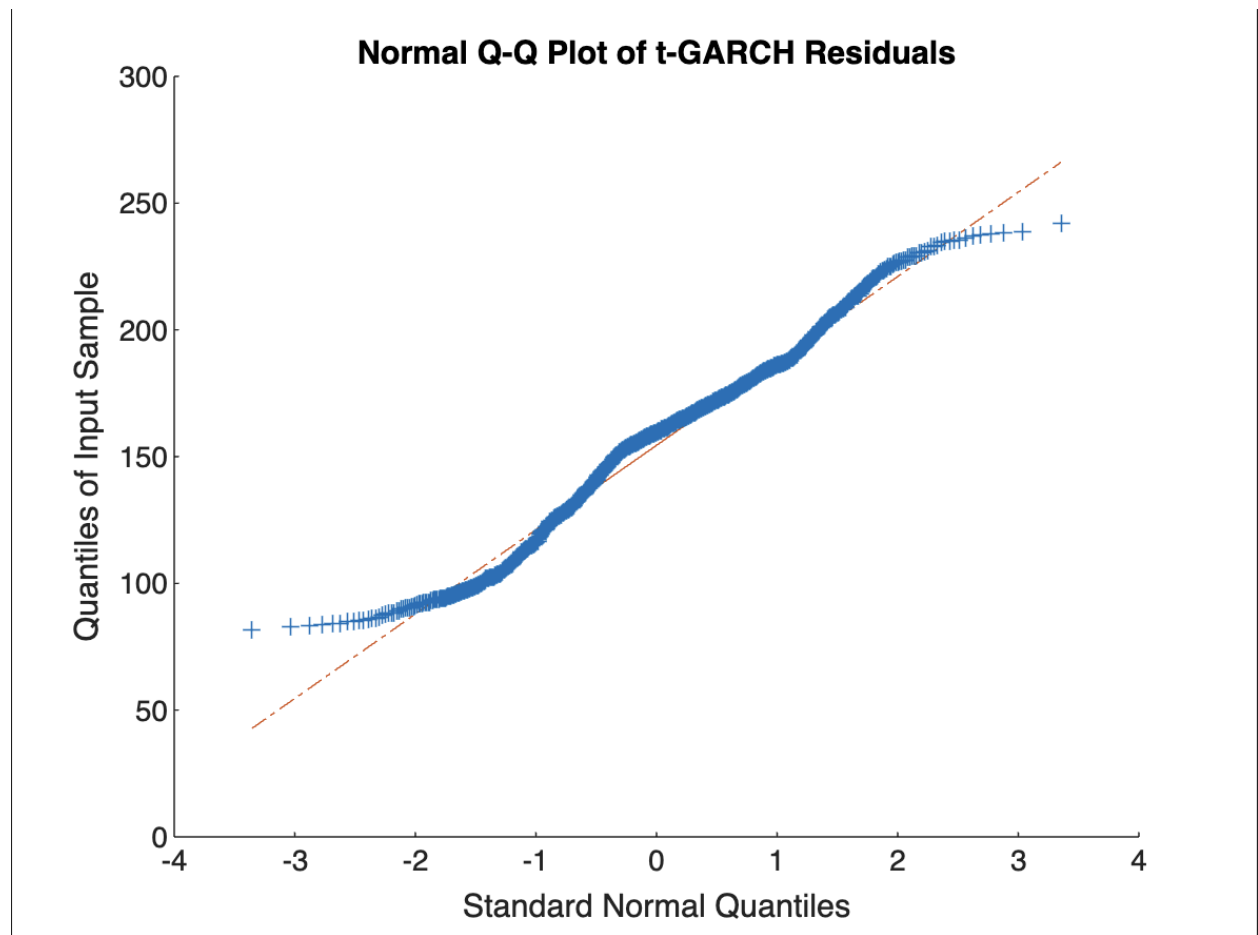
	Value	StandardError	TStatistic	PValue
Constant	2.1494e-05	3.9236e-06	5.4782	4.2957e-08
GARCH{1}	0.88608	0.016423	53.955	0
ARCH{1}	0.078535	0.015754	4.985	6.1966e-07
Leverage{1}	-0.0097798	0.019265	-0.50764	0.6117

GARCH with t-distribution:

GARCH(1,1) Conditional Variance Model (t Distribution):

	Value	StandardError	TStatistic	PValue
Constant	9.2084e-06	4.0764e-06	2.2589	0.023887
GARCH{1}	0.92595	0.018924	48.929	0
ARCH{1}	0.058088	0.015275	3.8028	0.0001431
DoF	4.9725	0.62842	7.9127	2.5181e-15

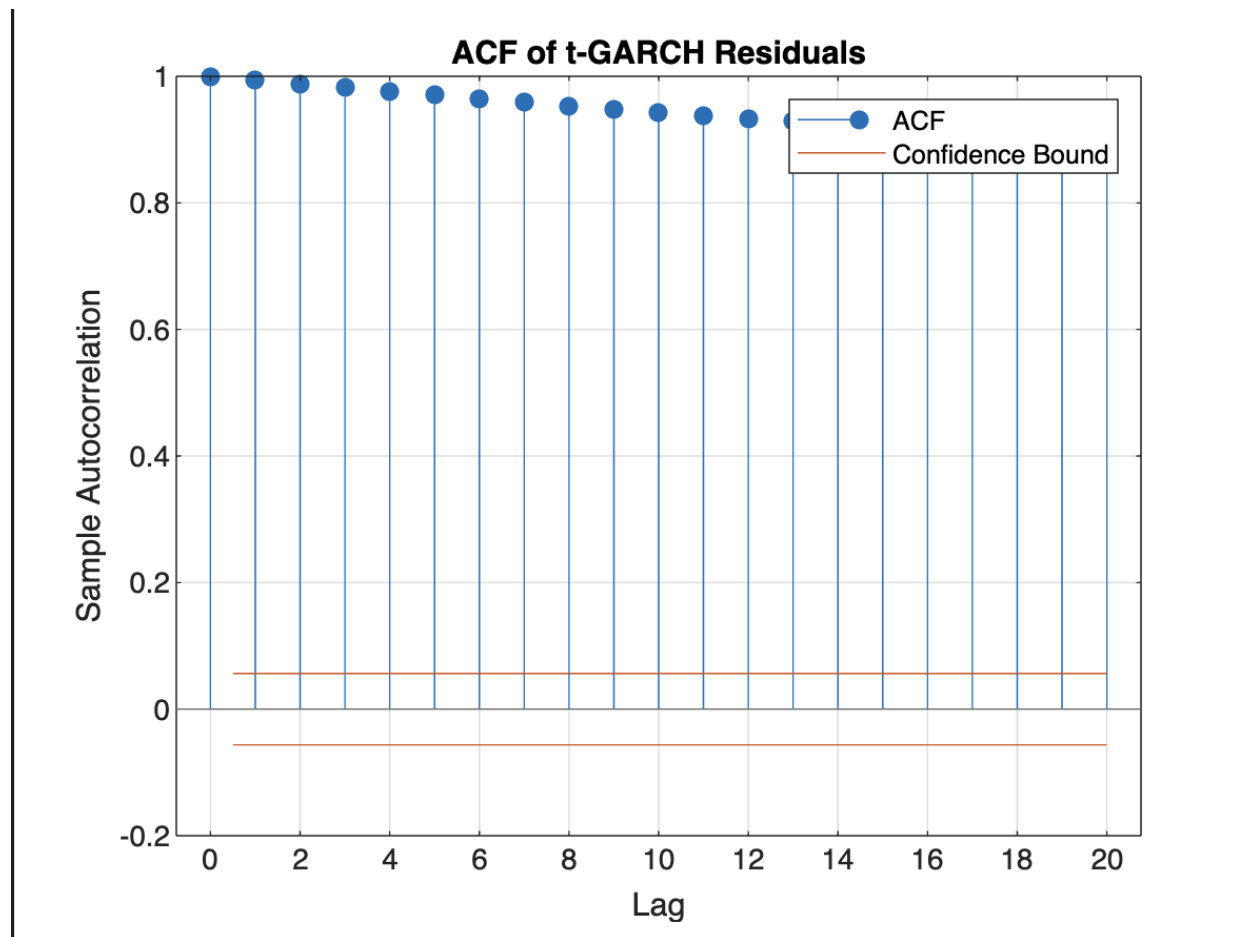
Normal Q-Q Plot of T-GARCH Residual:



Code:

```
% Normal Q-Q Plot  
qqplot(timeSeriesData);  
title('Normal Q-Q Plot of t-GARCH Residuals');
```

ACF of t-GARCH Residuals:



Code:

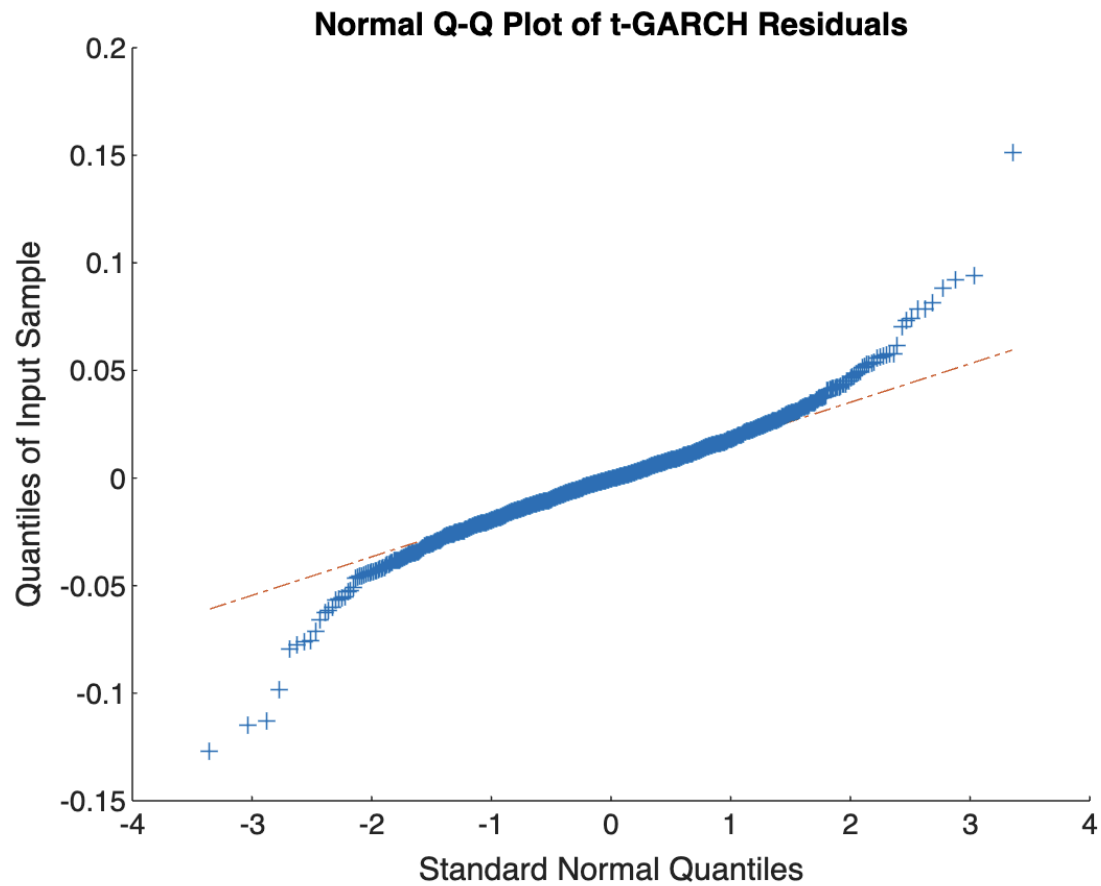
```
% Autocorrelation of residuals
```

```
figure;
```

```
autocorr(timeSeriesData);
```

```
title('ACF of t-GARCH Residuals');
```

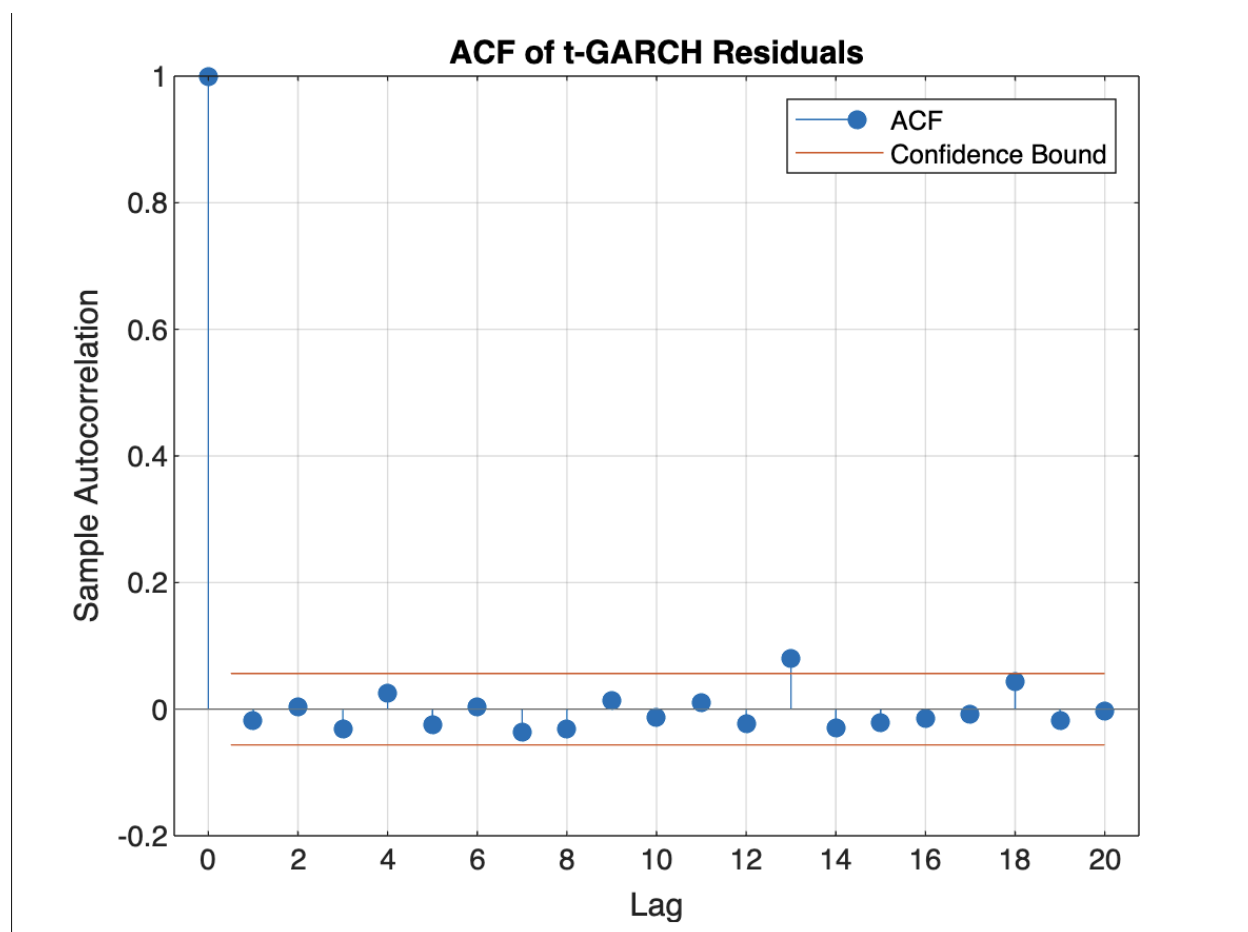
Normal Q-Q Plot of T-GARCH Residual (based of logReturns):



Code:

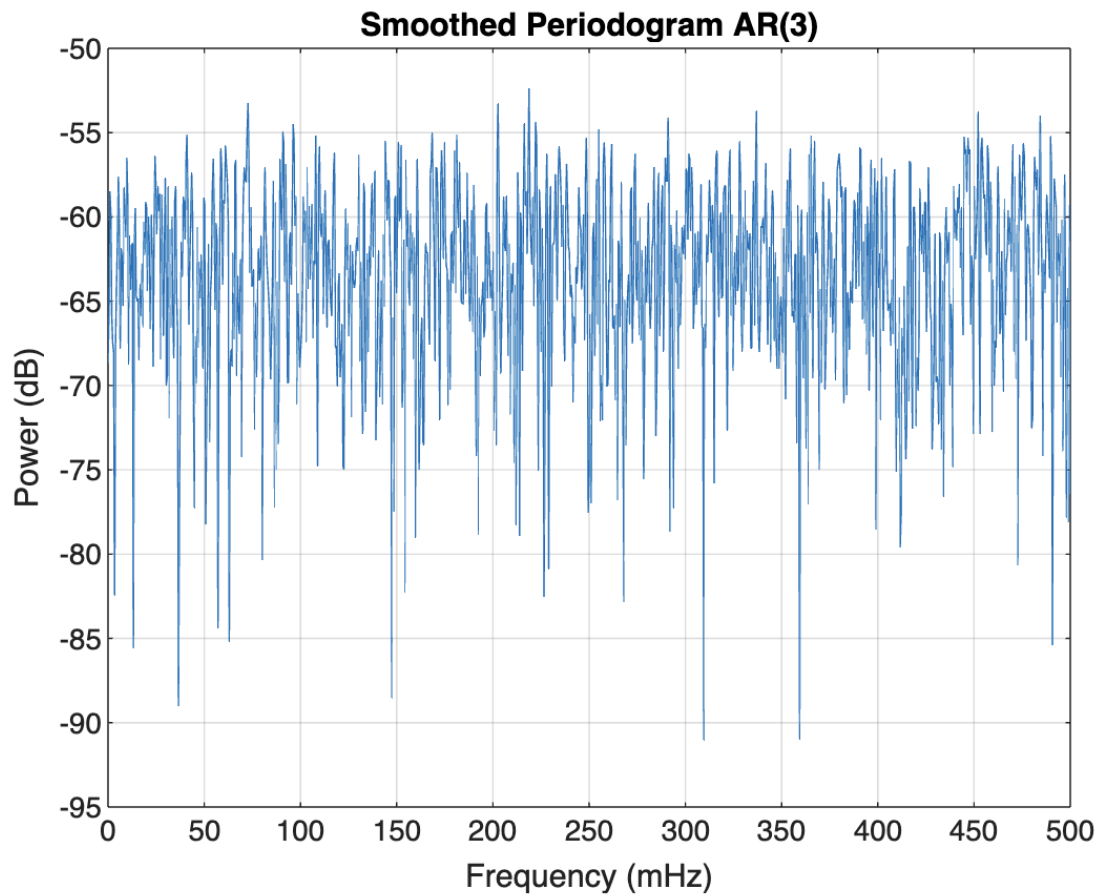
```
% Normal Q-Q Plot  
qqplot(logReturns);  
title('Normal Q-Q Plot of t-GARCH Residuals');
```

ACF of t-GARCH Residuals (based of logReturns):



Code:
figure;
autocorr(logReturns);
title('ACF of t-GARCH Residuals');

Smoothed Periodogram:



Code:

```
figure;  
periodogram(logReturns,[],[],[], 'power');  
title('Smoothed Periodogram AR(3)');
```

AIC Table for Model Comparison:

Model	AIC
-------	-----

{ 'ARMA(1,1)' }	-5957.8
{ 'ARMA(2,1)' }	-5956
{ 'ARMA(2,2)' }	-5954.6

Code:

```
models = {arima(1,0,1), arima(2,0,1), arima(2,0,2)};  
AIC_values = zeros(length(models),1);
```



```

for i = 1:length(models)
    fit = estimate(models{i}, logReturns, 'Display', 'off');
    [~,~,logL] = infer(fit, logReturns);
    numParams = length(fit.Constant) + length(fit.AR) + length(fit.MA) + 1; % +1 for
Variance
    AIC_values(i) = aicbic(logL, numParams);
end
T = table({'ARMA(1,1)';'ARMA(2,1)';'ARMA(2,2)'}, AIC_values, 'VariableNames',
{'Model', 'AIC'});
disp(T);

```

Model Fitting (ARMA(2,1)):

	Value	StandardError	TStatistic	PValue
Constant	-0.00060163	0.0010421	-0.57734	0.56371
AR{1}	-0.65089	0.46805	-1.3906	0.16434
AR{2}	0.012011	0.031444	0.38198	0.70247
MA{1}	0.63415	0.46969	1.3502	0.17697
Variance	0.00050464	1.1274e-05	44.761	0

Code:

```

model_21 = arima(2,0,1);
fit_21 = estimate(model_21, logReturns);

```

Inverse Roots ARMA(2,1) - Didn't work

Inverse Roots ARMA(2,2):

ARIMA(2,0,2) Model (Gaussian Distribution):

	Value	StandardError	TStatistic	PValue
Constant	-0.00012372	0.00030499	-0.40565	0.685
AR{1}	0.063939	0.60663	0.1054	0.91606
AR{2}	0.61128	0.36378	1.6804	0.092883
MA{1}	-0.096057	0.61115	-0.15717	0.87511
MA{2}	-0.60483	0.37159	-1.6277	0.10359
Variance	0.00050441	1.123e-05	44.915	0

Code:

```

model_22 = arima(2,0,2);
fit_22 = estimate(model_22, logReturns);
figure;
zplane(1, [1 -fit_22.AR{:}]);
title('Inverse Roots of ARMA(2,2)');

```

Likelihood Ratio Test between ARMA(2,2) and ARMA(2,1)L
Likelihood Ratio Test p-value: 0.4492

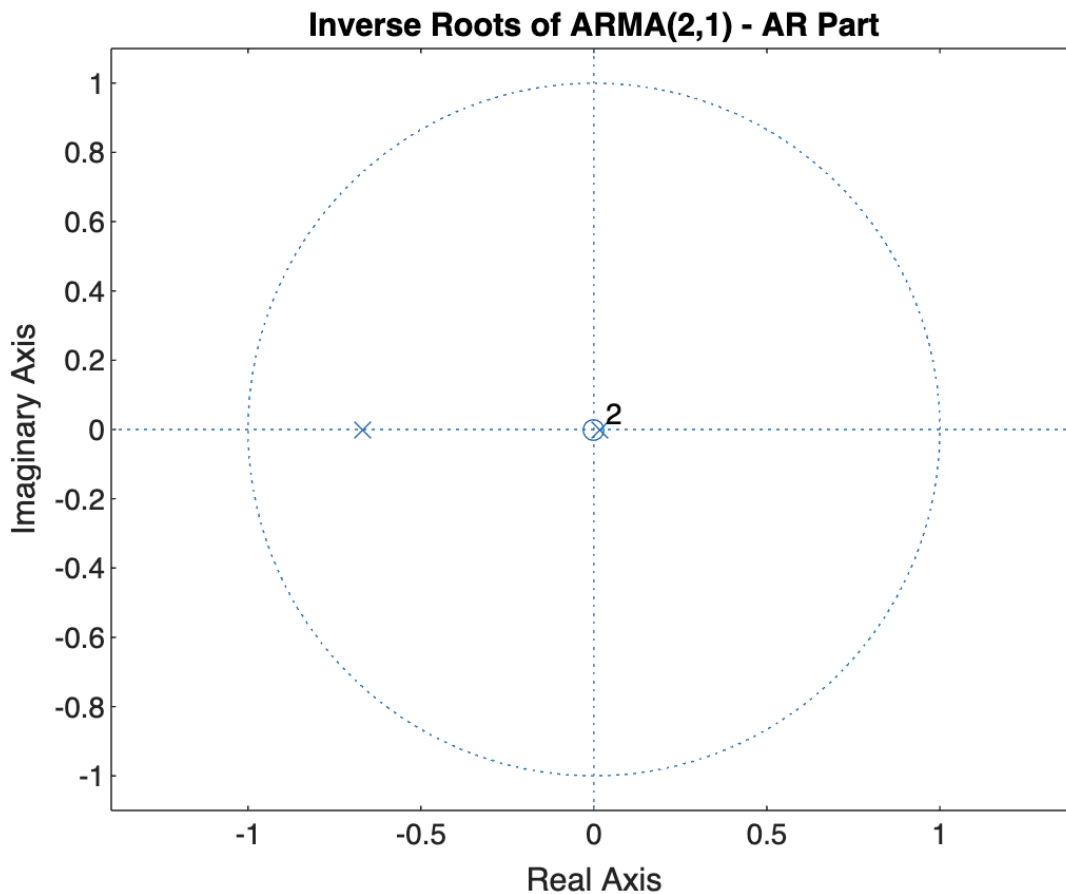
Code:

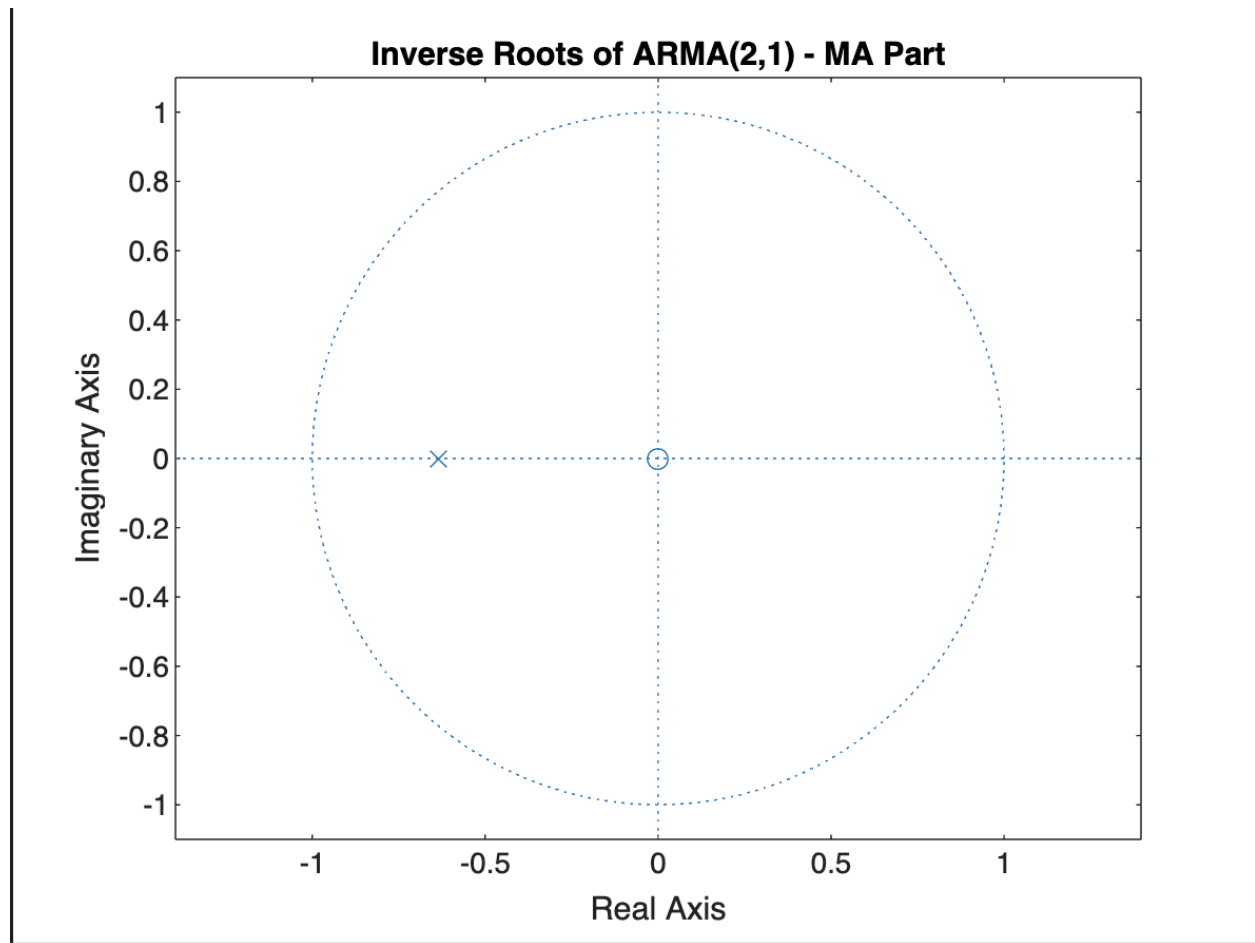
```

~,~,logL_22] = infer(fit_22, logReturns);
[~,~,logL_21] = infer(fit_21, logReturns);
LR_stat = -2*(logL_21 - logL_22);
p_val = 1 - chi2cdf(LR_stat, 1); % 1 additional MA parameter
fprintf('Likelihood Ratio Test p-value: %.4f\n', p_val);

```

Inverse Roots of ARMA(2,1) - AR Part and MA Part:





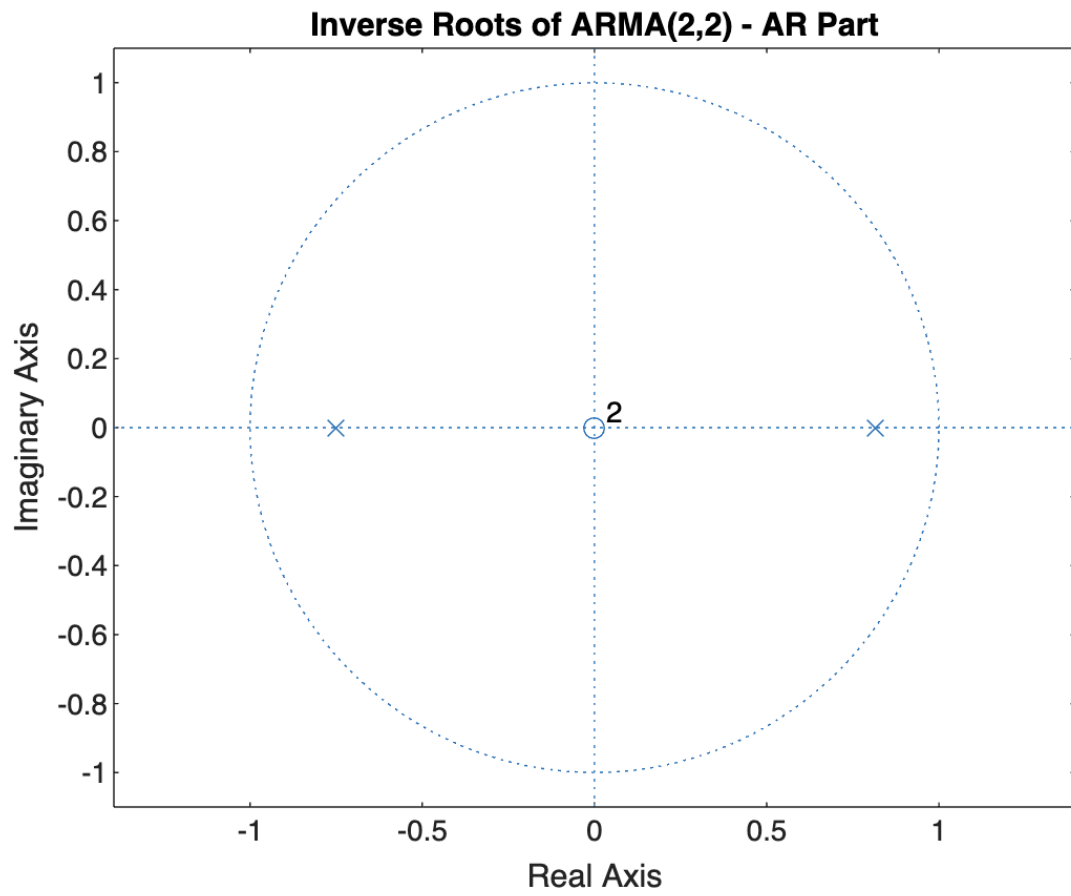
Code:

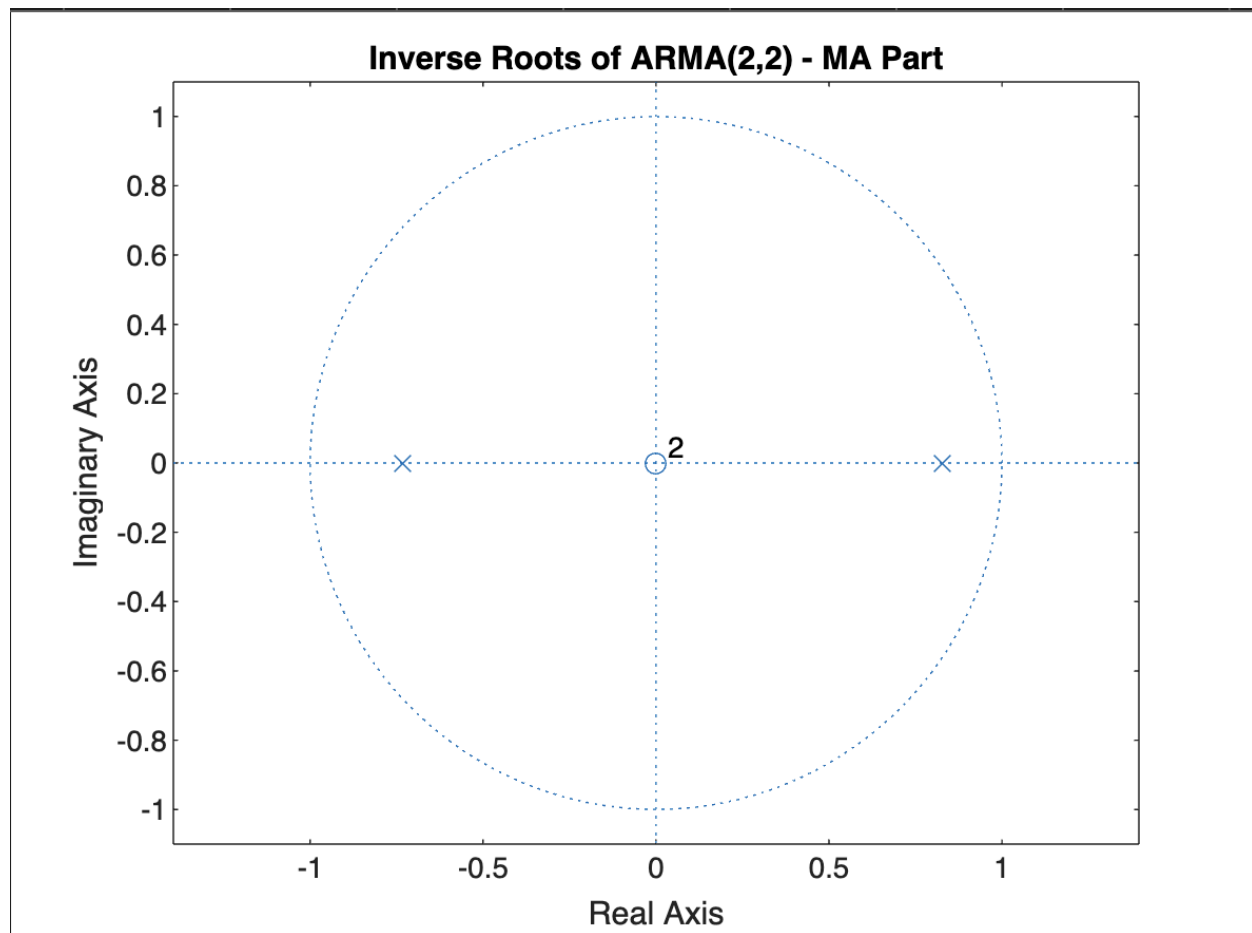
```
% Assume logReturns is already computed
model_21 = arima(2, 0, 1);
fit_21 = estimate(model_21, logReturns, 'Display', 'off');

% Plot AR roots (for stationarity)
figure;
zplane(1, [1, -cell2mat(fit_21.AR)]); % AR polynomial
title('Inverse Roots of ARMA(2,1) - AR Part');
xlabel('Real Axis'); ylabel('Imaginary Axis');

% Plot MA roots (for invertibility)
figure;
zplane(1, [1, cell2mat(fit_21.MA)]); % MA polynomial
title('Inverse Roots of ARMA(2,1) - MA Part');
xlabel('Real Axis'); ylabel('Imaginary Axis');
```

Inverse Roots of ARMA(2,2) - AR and MA Part:





Code:

```
model_22 = arima(2, 0, 2);
fit_22 = estimate(model_22, logReturns, 'Display', 'off');
```

% Plot AR roots

```
figure;
zplane(1, [1, -cell2mat(fit_22.AR)]);
title('Inverse Roots of ARMA(2,2) - AR Part');
xlabel('Real Axis'); ylabel('Imaginary Axis');
```

% Plot MA roots

```
figure;
zplane(1, [1, cell2mat(fit_22.MA)]);
title('Inverse Roots of ARMA(2,2) - MA Part');
xlabel('Real Axis'); ylabel('Imaginary Axis');
```

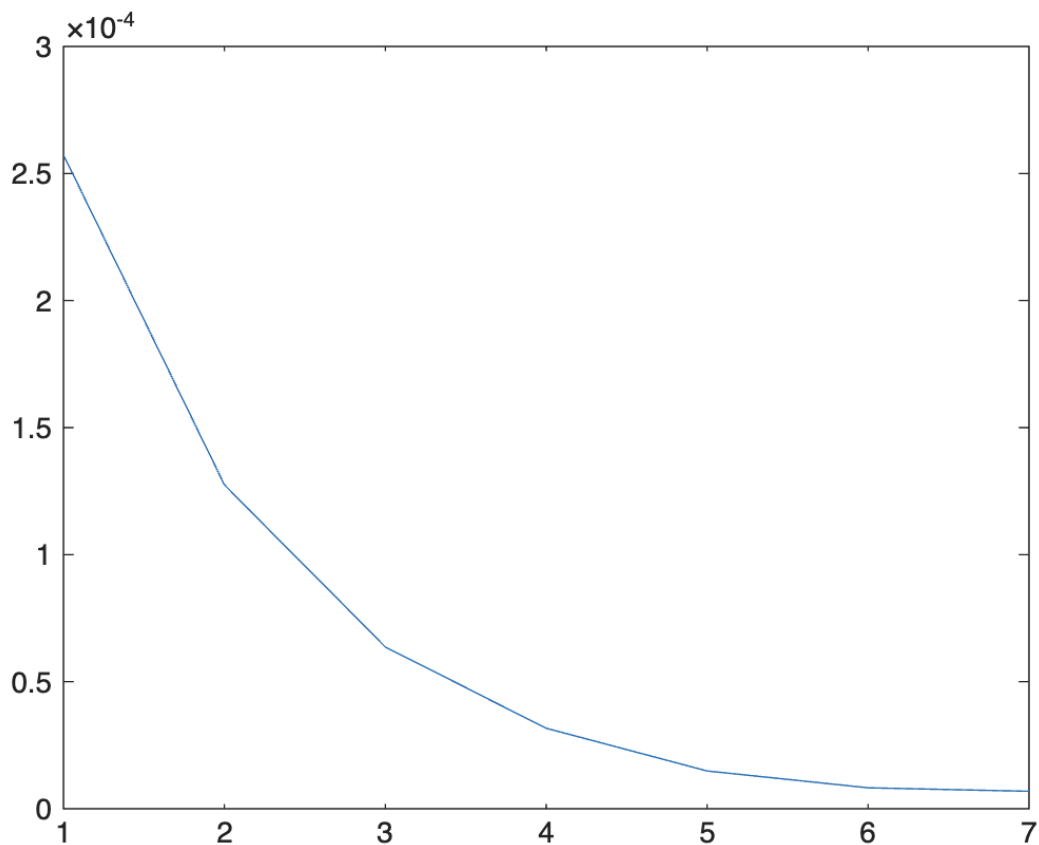
EGARCH(1,1) Conditional Variance Model (Gaussian Distribution):

	Value	StandardError	TStatistic	PValue
Constant	-0.032444	0.017832	-1.8194	0.068844
GARCH{1}	0.99538	0.0023858	417.21	0
ARCH{1}	0.061922	0.0075463	8.2056	2.2941e-16
Leverage{1}	-0.027469	0.0095519	-2.8758	0.0040305

Code:

```
model = egarch(1,1);  
fit = estimate(model, logReturns);
```

Wavelet-Based Volatility Analysis:



Code:

```
wt = modwt(logReturns);  
plot(modwtvar(wt)); % visualize scale-dependent variance
```

Sources:

<https://www.investopedia.com/terms/g/garch.asp>

<https://www.geeksforgeeks.org/arma-time-series-model/>

<https://www.machinelearningplus.com/time-series/augmented-dickey-fuller-test/>

<https://medium.com/pythoneers/stationarity-and-connectivity-using-graph-theory-to-uncover-temporal-patterns-in-time-series-data-ce4188f2b033>

<https://chatgpt.com/>

<https://matlab.mathworks.com/>

https://ionides.github.io/531w22/final_project/project22/blinded.html

https://ionides.github.io/531w24/final_project/project12/blinded.html

https://ionides.github.io/531w24/final_project/project07/blinded.html

https://ionides.github.io/531w24/final_project/project09/blinded.html

<https://gemini.google.com/app/db560369d8995133>

<https://ggplot2.tidyverse.org/reference/ggplot.html>

https://ionides.github.io/531w22/final_project/

<https://medium.com/@amit25173/time-series-analysis-for-stock-market-prediction-4954cdb39684>

RCode - AMZN

Exploratory Data Analysis:

ARIMA Model

GARCH Model

POMP Model

LSTM Model - <https://www.youtube.com/watch?v=oueuvXd4O9I&t=1678s>
https://www.r-bloggers.com/2021/04/lstm-network-in-r/#google_vignette

To use LSTMs in RStudio, you'll need to:

Install and load the necessary packages: Install keras and tensorflow using

`install.packages(c("keras", "tensorflow"))` and then load them with `library(keras)` and `library(tensorflow)`.

Prepare your data: This often involves reshaping your data into a suitable 3D format for LSTM input, which typically involves time steps, features, and batches.

Define your LSTM model: Use the `keras_model_sequential()` function to create a model, and add LSTM layers using `layer_lstm()` along with other layers as needed (e.g., dense layers for output).

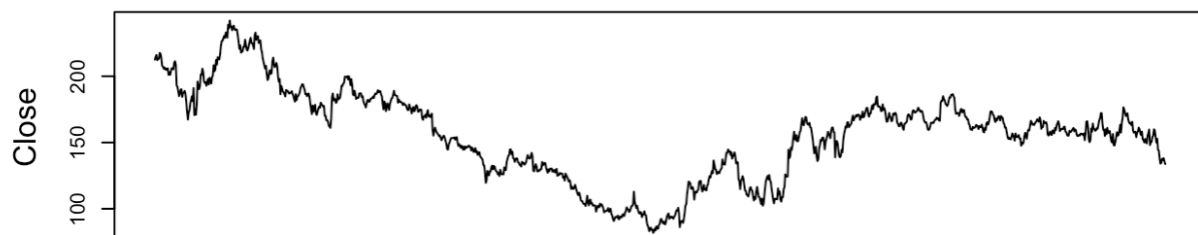
Compile the model: Specify the optimizer, loss function, and metrics for training.

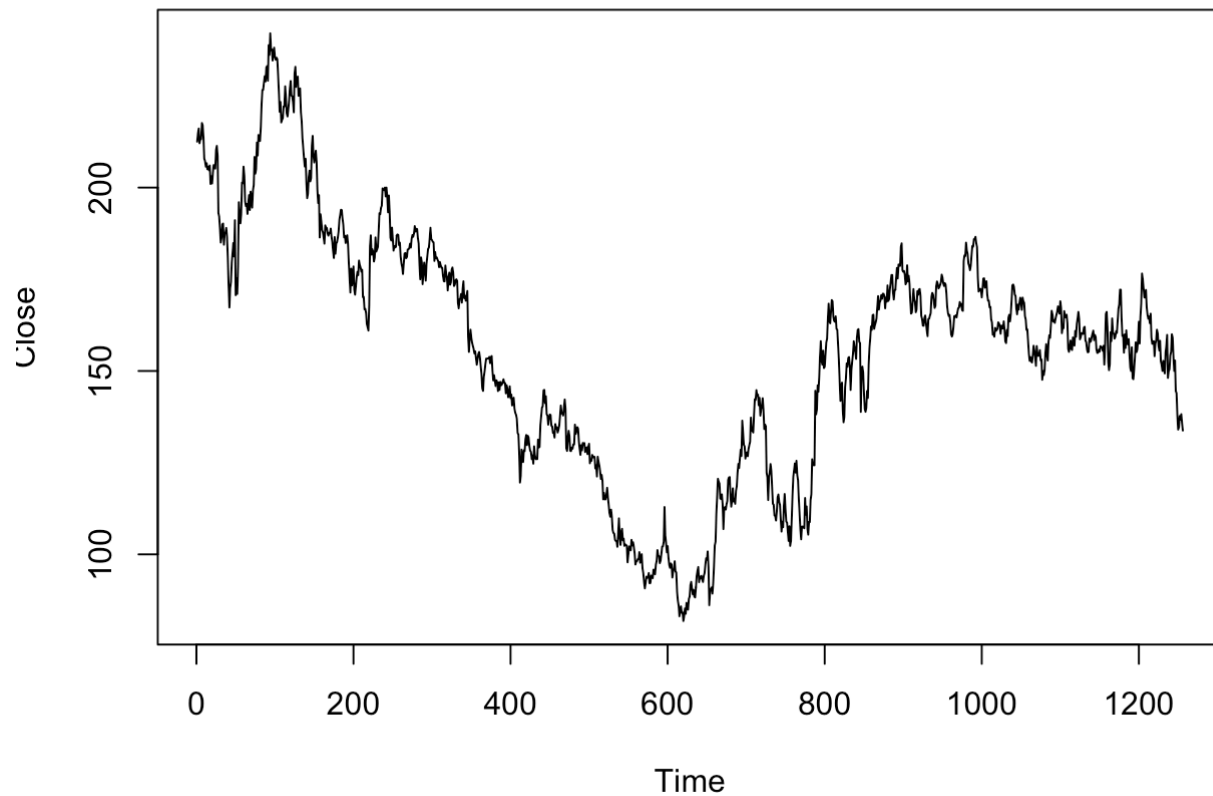
Train the model: Use the `fit()` function with your training data and validation data.

Evaluate and predict: Assess the model's performance using appropriate metrics and make predictions on new data.

Converter: [Online Matlab to R Converter](#)

Time Series Plot:





Code:

```
plot.ts(Close)
```

Code:

```
Amazon_Stock_Price_Data_Sigma_Scroll <-  
read.csv('Amazon_Stock_Price_Data_Sigma_Scroll', header = T, sep = ',')  
Amazon_Stock_Price_Data_Sigma_Scroll <-  
ts(Amazon_Stock_Price_Data_Sigma_Scroll, frequency = 12, start=c(2001, 1))  
Amazon_Stock_Price_Data_1_1_  
plot.ts(Amazon_Stock_Price_Data_Sigma_Scroll)
```

Link:

[▶ Plotting a Time Series graph on R Studio](#)

Step 1:

```
attach(FILE NAME)
```

Step 2:

```
library(tseries)
```

Step 3:

```
library(forecast)
```

Step 4:

```
plot.ts(Close)
```

Stationary (Dickey-Fuller Test): Test done in stats to check if a data set is stationary

Code: `> adf.test(Close)` - Check for stationary

Augmented Dickey-Fuller Test data: Close Dickey-Fuller = -1.8452, Lag order = 10,
p-value = 0.6439 alternative hypothesis: stationary

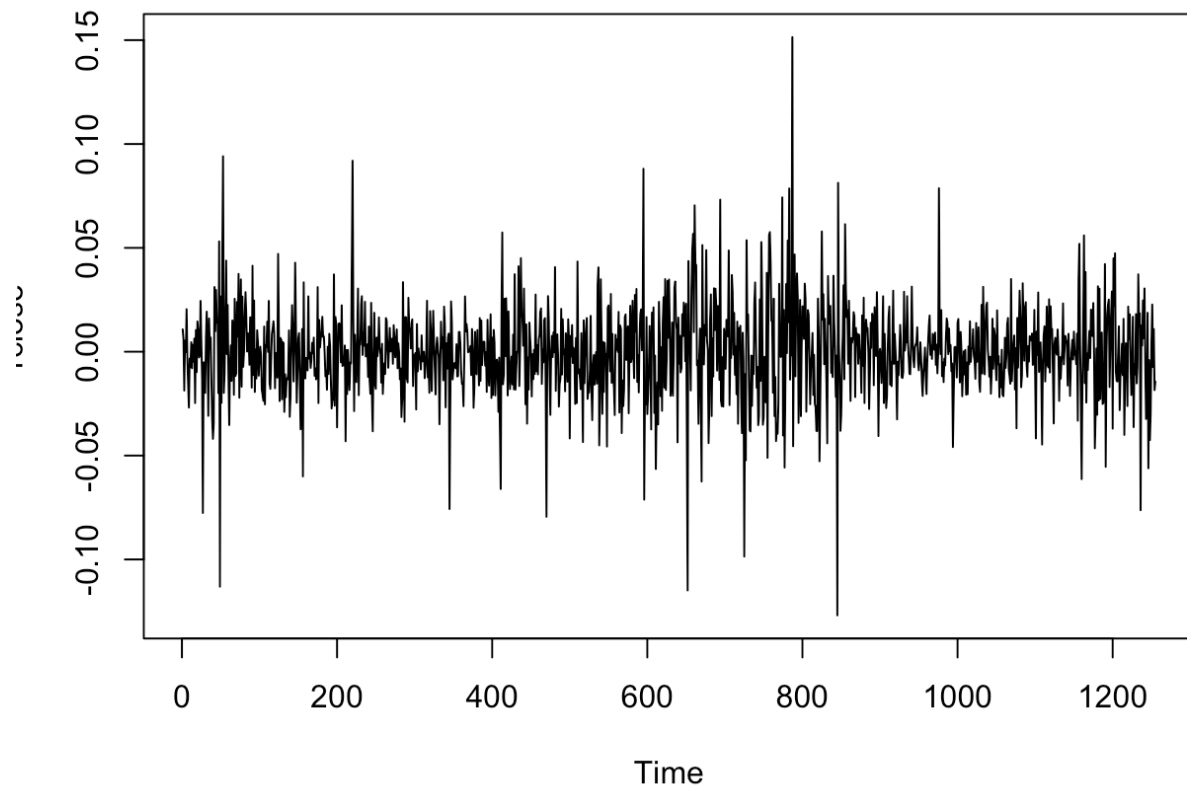
Since:

The p-value is 0.6439, which is much higher than 0.05,

We fail to reject the null hypothesis of the ADF test (which is: the series has a unit root),

Therefore, the series is non-stationary.

Log Return:



Code:

```
rclose=diff(log(Close))
> plot.ts(rclose)
```

Link: <https://www.youtube.com/watch?v=ny3gRhVsi4>

Second Stationary Test (Dickey Fuller Test):

Augmented Dickey-Fuller Test: This time the test is stationary as we have a p-value which is below or greater 0.05.

data: rclose

Dickey-Fuller = -11.087, Lag order = 10, p-value = 0.01

alternative hypothesis: stationary

In `adf.test(rclose)` : p-value smaller than printed p-value

Series: rclose

ARIMA(0,0,0) with zero mean

sigma^2 = 0.0005053: log likelihood = 2982.13
AIC=-5962.27 AICc=-5962.26 BIC=-5957.13

Results from model rclose:

Call:

arima(x = rclose, order = c(0, 0, 0))

Coefficients:

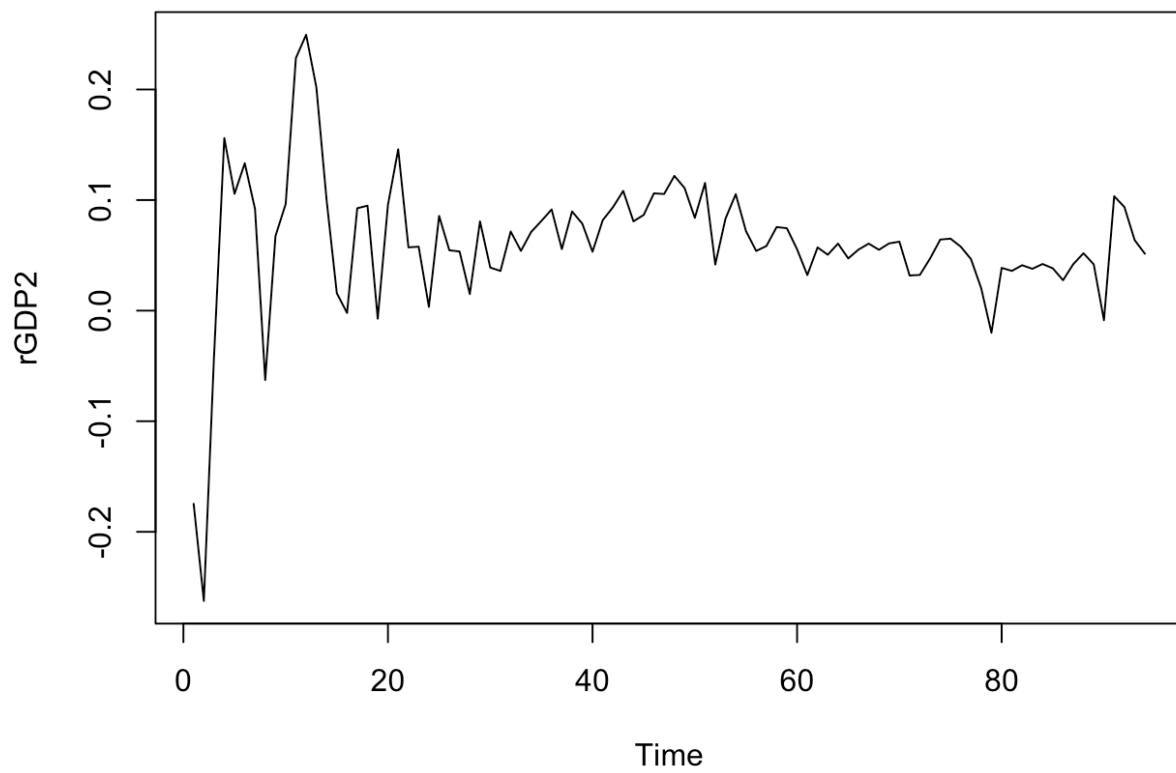
intercept

-4e-04

s.e. 6e-04

sigma^2 estimated as 0.0005052: log likelihood = 2982.3, aic = -5960.6

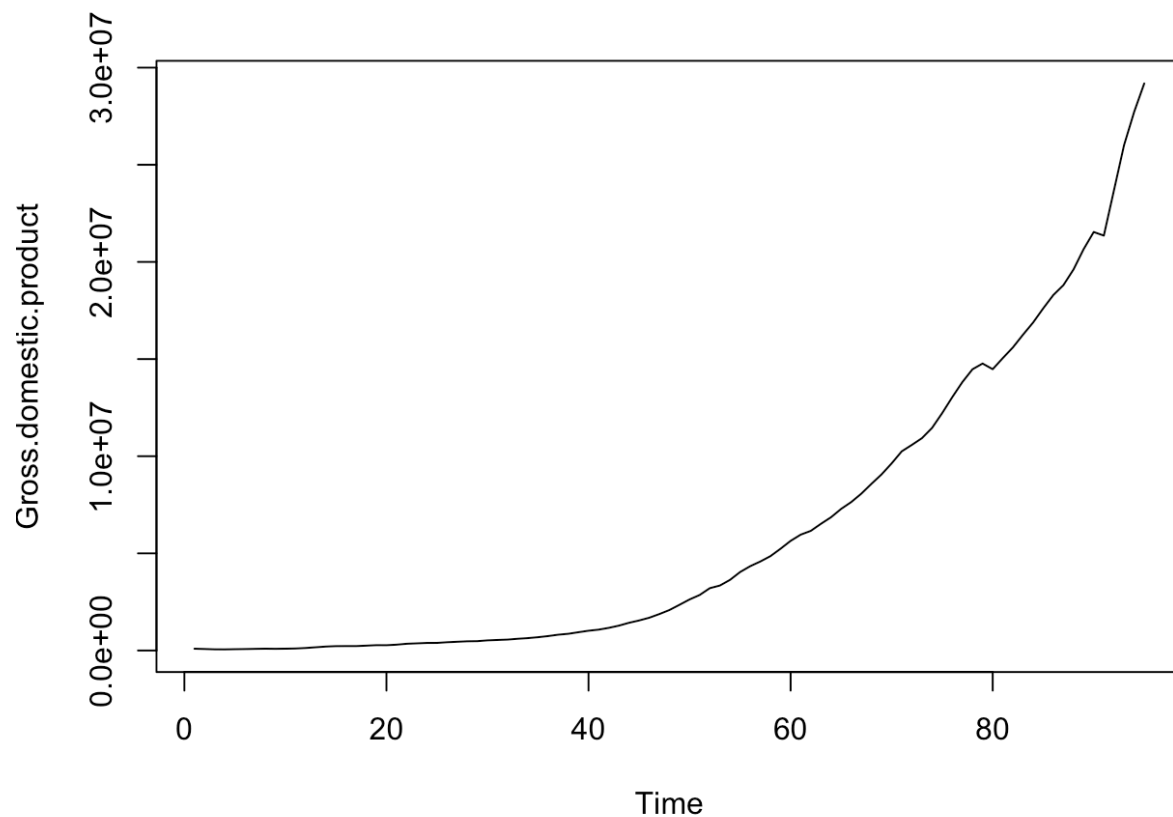
GDP - RCode



Remove stationary for GDP dataset:

```
> plot.ts(Gross.domestic.product)
> adf.test(Gross.domestic.product)
> plot.ts(rGD)
> attach(G2)
```


GDP Growth from 1930 to 2024:



Code:

```
rGDP2=diff(log(Gross.domestic.product))  
> adf.test(rGDP2)  
plot.ts(rGDP2)
```

ARIMA:

ARIMA(3,0,3) with non-zero mean

Coefficients:

	ar1	ar2	ar3	ma1	ma2	ma3	mean
	0.9741	-0.7386	-0.1618	-0.2979	0.3584	0.4239	0.0619
s.e.	0.3581	0.4190	0.3240	0.3414	0.2250	0.2016	0.0076

sigma^2 = 0.002264: log likelihood = 155.27
AIC=-294.54 AICc=-292.84 BIC=-274.19

Exploratory Data Analysis:

Min.	1st Qu.	Median	Mean	3rd Qu.	Max.
57154	389884	2081826	6139836	10416440	29184890

Gross.domestic.product	
nobs	9.500000e+01
NAs	0.000000e+00
Minimum	5.715400e+04
Maximum	2.918489e+07
1. Quartile	3.898835e+05
3. Quartile	1.041644e+07
Mean	6.139836e+06
Median	2.081826e+06
Sum	5.832844e+08
SE Mean	7.811825e+05
LCL Mean	4.588780e+06
UCL Mean	7.690892e+06
Variance	5.797337e+13
Stdev	7.614025e+06
Skewness	1.237608e+00
Kurtosis	4.882540e-01

ARCH-LM Test Results:

data: rGDP2

Chi-squared = 50.139, df = 12, p-value = 1.321e-06

Code:

```
> library(e1071)
> plot.ts(rGDP2)
> ArchTest(rGDP2)
```

GARCH Model Fit *

Conditional Variance Dynamics

GARCH Model : sGARCH(1,1)
Mean Model : ARFIMA(1,0,1)
Distribution : norm

Optimal Parameters

Estimate Std. Error t value Pr(>|t|)
mu -0.099880 0.043658 -2.2878 0.022150
ar1 0.994481 0.006388 155.6688 0.000000
ma1 -0.602452 0.114880 -5.2442 0.000000
omega 0.000109 0.000052 2.0992 0.035803
alpha1 0.505498 0.203760 2.4809 0.013107
beta1 0.465550 0.125739 3.7025 0.000213

Robust Standard Errors:

Estimate Std. Error t value Pr(>|t|)
mu -0.099880 0.030683 -3.2552 0.001133
ar1 0.994481 0.005916 168.0983 0.000000
ma1 -0.602452 0.108469 -5.5541 0.000000
omega 0.000109 0.000053 2.0510 0.040267
alpha1 0.505498 0.209818 2.4092 0.015987
beta1 0.465550 0.121230 3.8402 0.000123

LogLikelihood : 182.601

Information Criteria

Akaike -3.7575
Bayes -3.5951
Shibata -3.7650
Hannan-Quinn -3.6919

Weighted Ljung-Box Test on Standardized Residuals

statistic p-value
Lag[1] 0.5445 0.4606
Lag[2*(p+q)+(p+q)-1][5] 4.2762 0.0318
Lag[4*(p+q)+(p+q)-1][9] 5.8866 0.2755
d.o.f=2

HO : No serial correlation

Weighted Ljung-Box Test on Standardized Squared Residuals

 statistic p-value
Lag[1] 1.026 0.3112
Lag[2*(p+q)+(p+q)-1][5] 2.424 0.5221
Lag[4*(p+q)+(p+q)-1][9] 3.515 0.6719
d.o.f=2

Weighted ARCH LM Tests

 Statistic Shape Scale P-Value
ARCH Lag[3] 1.082 0.500 2.000 0.2982
ARCH Lag[5] 1.682 1.440 1.667 0.5458
ARCH Lag[7] 2.274 2.315 1.543 0.6594

Nyblom stability test

Joint Statistic: 1.0575
Individual Statistics:
mu 0.01549
ar1 0.06159
ma1 0.15983
omega 0.08441
alpha1 0.17229
beta1 0.22203

Asymptotic Critical Values (10% 5% 1%)

Joint Statistic: 1.49 1.68 2.12
Individual Statistic: 0.35 0.47 0.75

Sign Bias Test

 t-value prob sig
Sign Bias 0.10169 0.9192
Negative Sign Bias 0.31719 0.7518
Positive Sign Bias 0.06761 0.9463
Joint Effect 0.23680 0.9714

Adjusted Pearson Goodness-of-Fit Test:

```
-----  
group statistic p-value(g-1)  
1  20   24.72   0.16985  
2  30   34.94   0.20664  
3  40   60.89   0.01398  
4  50   60.26   0.13004
```

Elapsed time : 0.490967

```
> garch(rGDP2,grad="numerical",trace=FALSE)
```

Call:

```
garch(x = rGDP2, grad = "numerical", trace = FALSE)
```

Coefficient(s):

```
      a0      a1      b1  
0.0004715 0.3424809 0.5832913
```

Code for GARCH Test Results:

```
> rGDP2_garch_fit  
> garch(rGDP2,grad="numerical",trace=FALSE)
```

Equations for Final Project

POMP Model:

Feature	Global Search	Local Search
Goal	Explore broadly	Refine locally
Scope	Entire parameter space	Narrow region near a good estimate
Pros	Avoids local optima, robust	Faster convergence
Cons	Slower, computationally intensive	May converge to a local (not global) max
Example in R	<code>mif2()</code> with random <code>start</code>	<code>mif2()</code> using <code>coef()</code> from previous

Mean Equation (ARMA(1,1)):

$$r_t = \mu + \phi_1 r_{t-1} + \theta_1 \epsilon_{t-1} + \epsilon_t$$

- ♦ 1. ARIMA Model (AutoRegressive Integrated Moving Average)

An ARIMA(p, d, q) model includes:

AR(p): Autoregressive terms (past values)

I(d): Number of differences needed to make the series stationary

MA(q): Moving average terms (past errors)

General Equation:

$$\phi(B)(1 - B)^d Y_t = \theta(B)\varepsilon_t$$

Where:

- Y_t is the time series
- B is the backshift operator ($BY_t = Y_{t-1}$)
- $\phi(B) = 1 - \phi_1 B - \dots - \phi_p B^p$ (AR polynomial)
- $\theta(B) = 1 + \theta_1 B + \dots + \theta_q B^q$ (MA polynomial)
- $\varepsilon_t \sim N(0, \sigma^2)$ is white noise

Variance Equation (GARCH(1,1)):

$$\sigma_t^2 = \omega + \alpha_1 \varepsilon_{t-1}^2 + \beta_1 \sigma_{t-1}^2$$

♦ 2. GARCH Model (Generalized AutoRegressive Conditional Heteroskedasticity)

GARCH(p, q) models the **conditional variance** σ_t^2 .

Equations:

$$\begin{aligned}Y_t &= \mu + \varepsilon_t \\ \varepsilon_t &= \sigma_t z_t, \quad z_t \sim N(0, 1) \\ \sigma_t^2 &= \omega + \sum_{i=1}^q \alpha_i \varepsilon_{t-i}^2 + \sum_{j=1}^p \beta_j \sigma_{t-j}^2\end{aligned}$$

Where:

- $\omega > 0, \alpha_i \geq 0, \beta_j \geq 0$
- z_t is i.i.d. standard normal (can be replaced with t-distributed or skewed innovations)

GARCH

For financial time series modeling, generalized autoregressive conditional heteroskedasticity model, known as GARCH model, is widely used[7]. GARCH(p,q) model takes the following form:

$$Y_n = \epsilon_n \sqrt{V_n}$$

is the demeaned returns and where

$$V_n = \alpha_0 + \sum_{j=1}^q \alpha_j Y_{n-j}^2 + \sum_{k=1}^p \beta_k V_{n-k}$$

and ϵ_n is white noise.

When $p = 0$, it is a ARCH(q) model.

We use Garch(1,1), which is a popular choice[7] and has the form

$$Y_n = \epsilon_n \sqrt{V_n}$$

where

$$V_n = \alpha_0 + \alpha_1 Y_{n-1}^2 + \beta_1 V_{n-1}$$

and use AIC table to choose some other values of (p, q).

Hide

(Ex. from Ethereum Project)

♦ 3. POMP Model (Partially Observed Markov Process)

Used to model latent (unobserved) processes, often in epidemiology or macroeconomics.

General Structure:

- Latent process (state): X_t
- Observation process: Y_t

$$X_t \sim f(X_t | X_{t-1}, \theta) \quad (\text{State transition})$$

$$Y_t \sim g(Y_t | X_t, \theta) \quad (\text{Observation model})$$

- $f(\cdot)$: State process dynamics
- $g(\cdot)$: Measurement (observation) distribution
- θ : Parameters to be estimated

POMP

We learned that volatility can be modeled as a latent stochastic process, partially observed via the returns. And negative shocks will bring a subsequent increase in volatility, the phenomenon is called leverage. And we can use these assumptions to build a POMP model[7].

Model Structure

We try the model we learned from the lecture first[7], and we will use the results to change the model and analyze it further. We model the leverage R_n , the correlation between return on day $n-1$ and the increase in the log volatility from day $n-1$ to day n , as the following random walk on a transformed scale:

$$R_n = \frac{\exp(2G_n) - 1}{\exp(2G_n) + 1}$$

where G_n is a Gaussian random walk.

And the model is:

$$\begin{aligned} Y_n &= \exp(H_n/2)\epsilon_n \\ H_n &= \mu_h(1 - \phi) + \phi H_{n-1} + \beta_{n-1} R_n \exp(-H_{n-1}/2) + \omega_n \\ G_n &= G_{n-1} + v_n \end{aligned}$$

where Y_n is still the demeaned returns, $\beta_n = Y_n \sigma_{\eta} \sqrt{1 - \phi^2}$, $\{\epsilon_n\}$ is an i.i.d. $N(0, 1)$ sequence, $\{v_n\}$ is an i.i.d. $N(0, \sigma_v^2)$ sequence and $\{\omega_n\}$ is an i.i.d. $N(0, \sigma_{\omega,n}^2)$ sequence with $\sigma_{\omega,n}^2 = \sigma_{\eta}^2(1 - \phi^2)(1 - R_n^2)$. H_n is the log volatility. Then the latent state is (G_n, H_n) . [7][9]

Hide

Local Search:

◆ 4. Local Search (in POMP Models)

Local search optimizes parameters using gradient-based or likelihood maximization methods in a small neighborhood.

Common Objective:

$$\hat{\theta}_{\text{local}} = \arg \max_{\theta} \log p(Y_{1:T} | \theta)$$

- This is often done via **iterated filtering** or **simplex optimization** over a narrow range of initial conditions.

Global Search:

Global Search

Based on the results we get in our local search, we construct the box for the parameters as follows[12]:

$$\sigma_y = c(0.00001, 0.0025)$$

$$\mu_h = c(-7, -6)$$

$$\phi = c(0.7, 0.9)$$

$$\sigma_\eta = c(0.9, 1.1)$$

$$G_0 = c(-2, 2)$$

$$H_0 = c(-1, 1)$$

Hide

Simplified POMP Model:

Simplified POMP Model

Model Structure and Initial Simulation

In the last section, we find that σ_y will converge to 0, and G_0 will also converge to a small number, so we decide to simplify our POMP model and see the results.

We set $\sigma_y = 0$ and $G_0 = 0$, then since R_n will be 0, we can remove β in our model, then our model become this:

$$Y_n = \exp(H_n/2)\epsilon_n$$

$$H_n = \mu_h(1 - \phi) + \phi H_{n-1} + \omega_n$$

where $\{\omega_n\}$ is an i.i.d. $N(0, \sigma_\omega^2)$ sequence with $\sigma_\omega^2 = \sigma_\eta^2(1 - \phi^2)$.

Hide

◆ 5. Global Search (in POMP Models)

Global search finds optimal parameters over a large, potentially multimodal parameter space.

Objective Function:

$$\hat{\theta}_{\text{global}} = \arg \max_{\theta} \log p(Y_{1:T} | \theta)$$

Methods used include:

- Particle Swarm Optimization (PSO)
- Genetic Algorithms
- Grid or Latin Hypercube Sampling
- Random Search

Unlike local search, global methods **explore widely** and avoid local optima.

◆ 6. Simplified POMP Model

This is a reduced-form POMP model where assumptions are added to simplify:

Example (Linear Gaussian POMP):

$$\begin{aligned} X_t &= aX_{t-1} + w_t, & w_t &\sim N(0, \sigma_w^2) \\ Y_t &= bX_t + v_t, & v_t &\sim N(0, \sigma_v^2) \end{aligned}$$

This can be solved with a **Kalman Filter** for state estimation.

◆ 7. Force Negative POMP Model

This is a special POMP variation that **forces constraints** (e.g., negative drift or declining trend). Used in modeling decay or decline processes (e.g., economic shocks, population loss).

Example with Forced Negative Drift:

$$\begin{aligned} X_t &= X_{t-1} - \delta + w_t, & \delta &> 0 \\ Y_t &\sim \text{Poisson}(\lambda_t), & \lambda_t &= \max(X_t, 0) \end{aligned}$$

This imposes monotonic decay unless interrupted by  rise.

Plan for Final Project - GDP

Title

Modeling and Forecasting U.S. GDP Growth: A Time Series Analysis with ARIMA, GARCH, and POMP Models

Project Plan Outline

1. Introduction and Motivation (*Communicating Your Data Analysis*)

- **Goal:** Set the stage with a compelling narrative that frames GDP growth as a crucial metric for U.S. economic health, linking it to historical events (e.g., Great Depression, 2008 crisis, COVID-19).
- **Include:**
 - Why GDP growth is important.
 - Major macroeconomic shifts in the past 95 years.
 - Literature review of key models used in GDP time series forecasting.
 - Motivation to improve or reinterpret previous analyses using more modern or robust techniques like POMP.

- **Example Question to Raise:**

“Can nonlinear models like POMP more accurately capture the volatility and structural shifts in U.S. GDP growth compared to traditional ARIMA and GARCH models?”

2. Data Collection and Preprocessing

- **Dataset:** U.S. GDP growth (quarterly or annual, from 1929 to 2024) from BEA or FRED.
 - **Steps:**
 - Import data into RStudio from `.csv` or Excel.
 - Clean missing values, apply log transformations or differencing if necessary.
 - Convert to time series object (`ts`, `xts`, or `zoo`).
 - **Plot:**
 - Raw time series
 - Log-transformed or first differences
 - Overlay with major events (e.g., wars, crises)
-

3. Exploratory Time Series Analysis (ETS)

- **Plot:**
 - Time Series + Log Return
 - Decomposition (additive/multiplicative): trend, seasonality, noise
- **Test Stationarity:**
 - Augmented Dickey-Fuller Test (ADF)
 - KPSS Test
- **Autocorrelation & Partial Autocorrelation:**
 - ACF and PACF plots

4. Modeling and Methodology (*Statistical Methodology*)

♦ ARIMA Modeling

- Fit ARIMA(p,d,q) models (use `auto.arima()` as a benchmark)
- Test residuals for white noise (ACF + Ljung-Box)
- Model selection via AIC/BIC
- Forecast future GDP growth and evaluate accuracy

♦ GARCH Modeling

- Apply GARCH(1,1), then extend to:
 - **Asymmetric GARCH** (e.g., EGARCH, GJR)
 - **Student's t-distributed GARCH** (for heavy tails)
- Evaluate:
 - Residual diagnostics (ACF of residuals, Q-Q plots)
 - Compare volatility forecasts and error distributions

♦ POMP Modeling

- Begin with:
 - **Simplified GDP growth model** (e.g., linear Gaussian state-space)
 - Fit via `pomp` package
- Extend to:
 - **Force Negative POMP Model** (accounting for contraction periods)

- **Local Search:** iterated filtering, particle filtering
 - **Global Search:** MIF2, GenIF, random walk perturbations
 - Evaluate via:
 - Log-likelihood trajectories
 - AIC comparison table (ARIMA vs GARCH vs POMP variants)
-

5. Evaluation and Diagnostics

- **Model Comparison Table:**
 - AIC, BIC, RMSE across models
 - **Forecasting Performance:**
 - Out-of-sample predictions (2020–2024)
 - Visual comparison of forecast vs actual
 - **Residual Analysis:**
 - Residual plots
 - Normal Q-Q plots
 - ACF of residuals
-

6. Conclusions and Insights (*Reaching a Conclusion*)

- What model performed best? Why?
- Did nonlinear or probabilistic methods (like POMP) provide new insights?

- How do the forecasts compare with economic expectations or prior studies?
 - Limitations of the dataset or modeling assumptions.
 - Future directions: e.g., incorporating exogenous shocks (wars, pandemics) or regime-switching models.
-

7. Reproducibility and Scholarship (*Scholarship + Communication Style*)

- Include clear, referenced code snippets in appendices or Rmd
 - Provide summary tables instead of raw output
 - Cite all data sources (FRED, BEA), modeling papers (e.g., Box-Jenkins, GARCH papers, King et al. for POMP)
 - Compare and contrast with previous GDP time series studies
-

Deliverables Checklist

Item	Included?	Notes
Rmd file	✓	Cleanly annotated
Final Report PDF	✓	Organized with proper citations
AIC Table	✓	With explanation
All Code	✓	Submitted separately
References	✓	Cited in-text and in bibliography
Figures & Graphs	✓	Clean and interpretable

Teamwork
Statement



If working in a group

Final Tips for Publication-Readiness

1. **Tell a Story:** Frame your analysis around macroeconomic themes and historical relevance.
 2. **Be Critical:** Discuss assumptions, model limitations, and residual issues.
 3. **Make It Visual:** Clear plots comparing model fits, forecast intervals, etc.
 4. **Model Hierarchy:** Show why each new model adds value (ARIMA → GARCH → POMP).
 5. **Write with Clarity:** Explain intuition behind models for a general data science audience.
-

Completed

1. Data Preprocessing & Transformation

- Raw GDP data imported.
- Log transformation and differencing to obtain GDP growth (`rGDP2 = diff(log(GDP))`).
- Exploratory summary statistics printed.

2. Stationarity Testing

- ADF test on raw GDP → non-stationary.

- ADF test on `rGDP2` → stationary (confirmed).
- Time series plot of stationary GDP growth (`plot.ts(rGDP2)` shown and attached).

3. ARIMA Model

- ARIMA(3,0,3) fitted.
- Coefficients, standard errors, AIC, BIC, and log-likelihood reported.
- Model residuals assessed.

4. GARCH Model

- ARCH-LM test confirms volatility clustering.
- GARCH(1,1) model fitted with detailed diagnostics:
 - Coefficients
 - Robust SE
 - Log-likelihood and AIC
 - Ljung-Box and ARCH LM tests
 - Nyblom stability test
 - Sign bias test
 - Pearson GoF test

Partially Done

5. Visualization

- Plot of **rGDP2** (attached).
 - **Q-Q plot of GARCH residuals** (to check normality).
 - **Residual plot of ARIMA model** (should be white noise).
 - **Forecast plots** from ARIMA and GARCH.
 - Optional: overlay **historical events** (e.g., 2008, 2020).
-

Still Needed

6. Extended Models

- Fit **GARCH with t-distribution** or **EGARCH/GJR-GARCH** (for asymmetry).
 - Plot and interpret residuals of extended GARCH models.
 - Compare model fit using AIC/BIC.
-

7. POMP Models (Partially Observed Markov Process)

- Define a **state-space model** (latent GDP growth dynamics).
- Implement a **simplified POMP model** (e.g., local level).
- Implement a **"force negative" POMP model** (simulate contraction).
- Run **local search** (e.g., iterated filtering).
- Run **global search** (e.g., particle MCMC or random walk).
- Plot particle trajectories, likelihood surface, and forecast envelopes.

8. Model Comparison and Evaluation

- Create **comparison table**: ARIMA vs GARCH vs POMP (AIC, BIC, LL).
- Run **forecast evaluation**: RMSE, MAE, etc.
- Write interpretation: which model fits GDP growth best and why.

9. Final Report Writing

- **Introduction** (economic context + modeling motivation).
- **Methodology** section (explain time series approach).
- **Results and discussion** (interpret ARIMA, GARCH, POMP).
- **Conclusion** (GDP insights + forecast implications).
- **Clean code appendix** (in R, or PDF/HTML).
- Add **citations** for ADF, ARIMA, GARCH, POMP sources.

Summary Table

Section	Status
Data transformation	 Complete

Stationarity tests	 Complete
EDA summary	 Complete
ARIMA modeling	 Complete
GARCH modeling	 Complete
ACF/Q-Q/Residual plots	 Partial
GARCH extensions (EGARCH/t)	 Missing
POMP model & search methods	 Missing
Forecast plots	 Missing
Model comparison (AIC/BIC)	 Missing
Final report + interpretation	 Missing

Plan for Final Project - Bitcoin

```
> rBTC_Close_garch=ugarchspec(variance.model = list(garchOrder=c(1,1)),mean.model = list(c
> rBTC_Close_garch_fit=ugarchfit(rBTC_Close_garch,data = rBTC_Close)
> rBTC_Close_garch_fit
```

```
l,1)),mean.model = list(armaOrder=c(1,1)))
se)
```

```
> plot(rBTC_Close_garch_fit)
```

```
 = list(garchOrder=c(1,1)),mean.model = list(armaOrder=c(1,1)))
garch,data = rBTC_Close)
```

```
· garch(rBTC_Close,grad="numerical",trace=FALSE)
```

```

> par(mfrow=c(1,2),mai=c(0.8,0.8,0.1,0.3))
> dat <- read.table("BTC6.csv",sep=" ",header=TRUE)
> plot(as.Date(dat$Date),dat$Close,
+       xlab="date",ylab="BTC",type="l")
> plot(as.Date(dat$Date),dat$Close, log="y",
+       xlab="date",ylab="BTC",type="l")
> save(dat, file = "btc6.rda")
> btc6.ret <- diff(log(BTC6$Close))
> btc6.ret.demeaned <- btc6.ret - mean(btc6.ret)
> par(mai=c(0.8,0.8,0.1,0.1))
> load(file="btc6.rda")
> plot(btc6.ret.demeaned, type="l",
+       xlab="business day (2002-2012)",ylab="demeaned bitcoin return")
> library(tseries)
> library(rugarch)
> library(FinTS)
> library(e1071)

```

ArchTest(rBTC_Close)

```

BTC6 <- read.csv("~/POMP Model for AMZN Stock/AMZN_Stock/BTC6.csv")
View(BTC6)
attach(BTC6)

```

```

> par(mfrow=c(1,2),mai=c(0.8,0.8,0.1,0.3))
> dat <- read.table("BTC5.csv",sep=" ",header=TRUE)
>
> # *** IMPORTANT: Replace "%Y-%m-%d" with the actual format of your dates in BTC5.csv ***
> plot(as.Date(dat$Date, format = "%Y-%m-%d"), dat$Close,
+       xlab="date",ylab="BTC",type="l")

```

```

> par(mfrow=c(1,2),mai=c(0.8,0.8,0.1,0.3))
> dat <- read.table("BTC5.csv",sep=" ",header=TRUE)
> plot(as.Date(dat$Date),dat$Close,
+       xlab="date",ylab="BTC",type="l")

```

```
> set.seed(2050320976)
> par(mfrow=c(1,2),mai=c(0.8,0.8,0.1,0.3))
> dat <- read.table("BTC5.csv",sep="," ,header=TRUE)
> plot(as.Date(dat$Date),dat$Close,
+       xlab="date",ylab="BTC",type="l")
```

```
> BTC5 <- read.csv("~/POMP Model for AMZN Stock/AMZN_Stock/BTC5.csv")
> View(BTC5)
> attach(BTC5)
```

```
> set.seed(2050320976)
> par(mfrow=c(1,2),mai=c(0.8,0.8,0.1,0.3))
> dat <- read.table("BTC4.csv",sep="," ,header=TRUE)
> plot(as.Date(dat$Date),dat$Close,
+       xlab="date",ylab="BTC",type="l")
```

· `attach(BTC4)`

```
> BTC4 <- read.csv("~/POMP Model for AMZN Stock/AMZN_Stock/BTC4.csv")
> View(BTC4)
>       xlab="date",ylab="BTC",type="l")
```

```
> BTC2 <- read.csv("~/POMP Model for AMZN Stock/AMZN_Stock/BTC2.csv", header=FALSE, comment.char="#")

Warning messages:
1: In grep("^[^#].*", lines, value = TRUE) :
  unable to translate 'Date,Open,High,Low,Close<a0>,Adj Close<a0>' to a wide string
2: In grep("^[^#].*", lines, value = TRUE) : input string 1 is invalid

> View(BTC2)
> BTC3 <- read.csv("~/POMP Model for AMZN Stock/AMZN_Stock/BTC3.csv", comment.char="#")
```

", header=FALSE, comment.char="#")

```
>
> dat <- read.table("BTC2.csv",sep="," ,header=TRUE)
```

```
Error in make.names(col.names, unique = TRUE) :
  invalid multibyte string 5
```

```
> dat <- read.table("BTC2.csv",sep="," ,header=TRUE)
```

```
Error in make.names(col.names, unique = TRUE) :
```

```
> View(BTC2)
> attach(BTC2)
```

The following objects are masked from BTC:

V1, V2, V3, V4, V5, V6

```
> par(mfrow=c(1,2),mai=c(0.8,0.8,0.1,0.3))
> dat <- read.table("BTC2.csv",sep="," ,header=TRUE)
```

```
> plot(as.Date(dat$Date),dat$Close,  
+       xlab="date",ylab="BTC",type="l")
```

```
Error in plot.window(...) : need finite 'xlim' values
```

In addition: **Warning messages:**

```
1: In min(x) : no non-missing arguments to min; returning Inf  
2: In max(x) : no non-missing arguments to max; returning -Inf  
3: In min(x) : no non-missing arguments to min; returning Inf  
4: In max(x) : no non-missing arguments to max; returning -Inf
```

```
> BTC2 <- read.csv("~/POMP Model for AMZN Stock/AMZN_Stock/BTC2.csv", header=FALSE,
```

header=FALSE, comment.char="#")

```
> plot.ts(Close)  
> adf.test(Close)
```

Augmented Dickey-Fuller Test

data: Close

Dickey-Fuller = 2.3068, Lag order = 28, p-value = 0.99

alternative hypothesis: stationary

```
> View(BTC)  
> attach(BTC);  
> library(tseries)  
> library(forecast)
```

```
(("~/POMP Model for AMZN Stock/AMZN_Stock/BTC.csv", header=FALSE, comment.char="#")
```

```
> BTC <- read.csv("~/POMP Model for AMZN Stock/AMZN_Stock/BTC.csv", header=FALSE,
```

```
> my_stan_model <- stan_model("my_model.stan");
```

```
fit <- sampling(my_stan_model, data = stan_data, algorithm = "Fixed_param", iter = 1000,
```

```
, iter = 1000, chains = 1)
```

```
> Y_state = normal_rng( 0,exp(H/2) );
```

```
Error in normal_rng(0, exp(H/2)) : could not find function 'normal_rng'
```

```
> Y_state = rnorm( 0,exp(H/2) );
```

```
Error: object 'H' not found
```

```
> my_stan_model <- stan_model("my_model.stan");
```

```
SAMPLING FOR MODEL 'anon_model' NOW (CHAIN 1).
Chain 1: Iteration: 1 / 1000 [ 0%] (Sampling)
Chain 1: Iteration: 100 / 1000 [ 10%] (Sampling)
Chain 1: Iteration: 200 / 1000 [ 20%] (Sampling)
Chain 1: Iteration: 300 / 1000 [ 30%] (Sampling)
Chain 1: Iteration: 400 / 1000 [ 40%] (Sampling)
Chain 1: Iteration: 500 / 1000 [ 50%] (Sampling)
Chain 1: Iteration: 600 / 1000 [ 60%] (Sampling)
Chain 1: Iteration: 700 / 1000 [ 70%] (Sampling)
Chain 1: Iteration: 800 / 1000 [ 80%] (Sampling)
Chain 1: Iteration: 900 / 1000 [ 90%] (Sampling)
Chain 1: Iteration: 1000 / 1000 [100%] (Sampling)
Chain 1:
Chain 1: Elapsed Time: 0 seconds (Warm-up)
Chain 1: 0.001 seconds (Sampling)
Chain 1: 0.001 seconds (Total)
Chain 1:
```

```
> fit <- sampling(my_stan_model, data = stan_data, algorithm = "Fixed_param",
```

```
> Y_state = normal_rng( 0,exp(H/2) );
```

```
Error in normal_rng(0, exp(H/2)) : could not find function "normal_rng"
```

```
> Y_state = rnorm( 0,exp(H/2) );
```

```
Error: object 'H' not found
```

```
> my_stan_model <- stan_model("my_model.stan");
```



```
SAMPLING FOR MODEL 'anon_model' NOW (CHAIN 1).
Chain 1: Iteration: 1 / 1000 [ 0%] (Sampling)
Chain 1: Iteration: 100 / 1000 [ 10%] (Sampling)
Chain 1: Iteration: 200 / 1000 [ 20%] (Sampling)
Chain 1: Iteration: 300 / 1000 [ 30%] (Sampling)
Chain 1: Iteration: 400 / 1000 [ 40%] (Sampling)
Chain 1: Iteration: 500 / 1000 [ 50%] (Sampling)
Chain 1: Iteration: 600 / 1000 [ 60%] (Sampling)
Chain 1: Iteration: 700 / 1000 [ 70%] (Sampling)
Chain 1: Iteration: 800 / 1000 [ 80%] (Sampling)
Chain 1: Iteration: 900 / 1000 [ 90%] (Sampling)
Chain 1: Iteration: 1000 / 1000 [100%] (Sampling)
Chain 1:
Chain 1: Elapsed Time: 0 seconds (Warm-up)
Chain 1: 0.001 seconds (Sampling)
Chain 1: 0.001 seconds (Total)
Chain 1:
```

```
> fit <- sampling(my_stan_model, data = stan_data, algorithm = "Fixed_param",
```

```
, iter = 1000, chains = 1)
```

[[1]]

Stan model 'anon_model' does not contain samples.

[[2]]

Stan model 'anon_model' does not contain samples.

[[3]]

Stan model 'anon_model' does not contain samples.

[[4]]

Stan model 'anon_model' does not contain samples.

SAMPLING FOR MODEL 'anon_model' NOW (CHAIN 1).

[1] "Error : Must use algorithm=\"Fixed_param\" for model that has no parameters."
error occurred during calling the sampler; sampling not done

SAMPLING FOR MODEL 'anon_model' NOW (CHAIN 2).

[1] "Error : Must use algorithm=\"Fixed_param\" for model that has no parameters."
error occurred during calling the sampler; sampling not done

SAMPLING FOR MODEL 'anon_model' NOW (CHAIN 3).

[1] "Error : Must use algorithm=\"Fixed_param\" for model that has no parameters."
error occurred during calling the sampler; sampling not done

SAMPLING FOR MODEL 'anon_model' NOW (CHAIN 4).

[1] "Error : Must use algorithm=\"Fixed_param\" for model that has no parameters."
error occurred during calling the sampler; sampling not done

```

> stan_data <- list(
+   Y_state = 0.5, # Example value
+   sigma_eta = 0.1, # Example value
+   phi = 0.9,      # Example value
+   G_initial = 0.0, # Example initial G
+   mu_h = 0.0,     # Example value
+   H_initial = 0.0, # Example initial H
+   sigma_nu = 0.05 # Example value
+ )
> fit <- sampling(my_stan_model, data = stan_data, iter = 2000, chains = 4)
starting worker pid=63839 on localhost:11246 at 17:15:01.716
starting worker pid=63852 on localhost:11246 at 17:15:01.836
starting worker pid=63865 on localhost:11246 at 17:15:01.950
starting worker pid=63878 on localhost:11246 at 17:15:02.063

```

```

> my_stan_model <- stan_model("my_model.stan");

```

iter = 1000, chains = 1)

```

> data <- read.csv("BTC.csv", fileEncoding = "UTF-8", check.names = FALSE)

```

Warning messages:

- 1: In read.table(file = file, header = header, sep = sep, quote = quote, :
invalid input found on input connection 'BTC.csv'
- 2: In read.table(file = file, header = header, sep = sep, quote = quote, :
incomplete final line found by readTableHeader on 'BTC.csv'

```

> par(mfrow=c(1,2),mai=c(0.8,0.8,0.1,0.3))
> dat <- read.csv("BTC.csv",fileEncoding = "UTF-8", check.names = FALSE)

```

```
> modelrBTC_Close=arima(rBTC_Close, order = c(5,1,0))
> modelrBTC_Close
```

Call:

```
arima(x = rBTC_Close, order = c(5, 1, 0))
```

Coefficients:

	ar1	ar2	ar3	ar4	ar5
	-0.8969	-0.6686	-0.4902	-0.3314	-0.1220
s.e.	0.0064	0.0083	0.0088	0.0083	0.0064

sigma^2 estimated as 197.3: log likelihood = -97975.31, aic = 195962.6

```
> set.seed(2050320976)
> par(mfrow=c(1,2),mai=c(0.8,0.8,0.1,0.3))
> dat <- read.table("BTC.csv",sep=" ",header=TRUE)
```

```
> auto.arima(rBTC_Close)
```

Series: rBTC_Close

ARIMA(5,1,0)

Coefficients:

	ar1	ar2	ar3	ar4	ar5
	-0.8969	-0.6686	-0.4902	-0.3314	-0.1220
s.e.	0.0064	0.0083	0.0088	0.0083	0.0064

sigma^2 = 197.3: log likelihood = -97975.31

AIC=195962.6 AICc=195962.6 BIC=196011.2

```

      class = "nctest" )
}
<bytecode: 0x119bcf158>
<environment: namespace:tseries>
> adf.test(rBTC_Close)

```

Augmented Dickey-Fuller Test

```

data:  rBTC_Close
Dickey-Fuller = -30.613, Lag order = 28, p-value = 0.01
alternative hypothesis: stationary

```

```

> rBTC_Close=diff(Close);
> plot.ts(rBTC_Close)
> adf.test
function (x, alternative = c("stationary", "explosive"), k = trunc((length(x) -
  1)^(1/3)))
{
  if ((NCOL(x) > 1) || is.data.frame(x))
    stop("x is not a vector or univariate time series")
  if (any(is.na(x)))
    stop("NAs in x")
  if (k < 0)
    stop("k negative")
  alternative <- match.arg(alternative)
  DNAME <- deparse(substitute(x))
  k <- k + 1
  x <- as.vector(x, mode = "double")
  y <- diff(x)

```

```

n <- length(y)
z <- embed(y, k)
yt <- z[, 1]
xt1 <- x[k:n]
tt <- k:n
if (k > 1) {
  yt1 <- z[, 2:k]
  res <- lm(yt ~ xt1 + 1 + tt + yt1)
}
else res <- lm(yt ~ xt1 + 1 + tt)
res.sum <- summary(res)
STAT <- res.sum$coefficients[2, 1]/res.sum$coefficients[2,
  2]
table <- cbind(c(4.38, 4.15, 4.04, 3.99, 3.98, 3.96), c(3.95,
  3.8, 3.73, 3.69, 3.68, 3.66), c(3.6, 3.5, 3.45, 3.43,
  3.42, 3.41), c(3.24, 3.18, 3.15, 3.13, 3.13, 3.12), c(1.14,
  1.19, 1.22, 1.23, 1.24, 1.25), c(0.8, 0.87, 0.9, 0.92,
  0.93, 0.94), c(0.5, 0.58, 0.62, 0.64, 0.65, 0.66), c(0.15,
  0.24, 0.28, 0.31, 0.32, 0.33))
table <- -table
tablen <- dim(table)[2]
tableT <- c(25, 50, 100, 250, 500, 1e+05)
tablep <- c(0.01, 0.025, 0.05, 0.1, 0.9, 0.95, 0.975, 0.99)
tableipl <- numeric(tablen)
for (i in (1:tablen)) tableipl[i] <- approx(tableT, table[,
  i], n, rule = 2)$y
interpol <- approx(tableipl, tablep, STAT, rule = 2)$y
if (!is.na(STAT) && is.na(approx(tableipl, tablep, STAT,
  rule = 1)$y))
  if (interpol == min(tablep))
    warning("p-value smaller than printed p-value")
  else warning("p-value greater than printed p-value")
if (alternative == "stationary")
  PVAL <- interpol

```

```

else if (alternative == "explosive")
  PVAL <- 1 - interpol
else stop("irregular alternative")
PARAMETER <- k - 1
METHOD <- "Augmented Dickey-Fuller Test"
names(STAT) <- "Dickey-Fuller"
names(PARAMETER) <- "Lag order"
structure(list(statistic = STAT, parameter = PARAMETER, alternative = alternative,
  p.value = PVAL, method = METHOD, data.name = DNAME),
  class = "htest")
}
<bytecode: 0x119bcf158>
<environment: namespace:tseries>
> adf.test(rBTC_Close)

```

```
> adf.test(rBTC_Close)
```

Augmented Dickey-Fuller Test

```

data:  rBTC_Close
Dickey-Fuller = -30.613, Lag order = 28, p-value = 0.01
alternative hypothesis: stationary

```

```

*-----*
*           GARCH Model Fit           *
*-----*

```

Conditional Variance Dynamics

```

-----
GARCH Model      : sGARCH(1,1)
Mean Model       : ARFIMA(1,0,1)
Distribution      : norm

```

Optimal Parameters

```

-----
      Estimate  Std. Error  t value  Pr(>|t|)
mu      0.007472    0.001713    4.3630  0.000013
ar1     -0.134144    0.086768   -1.5460  0.122103
ma1      0.203012    0.085720    2.3683  0.017870
omega    0.000254    0.000033   7.7651  0.000000

```


alpha1	0.077659	0.003419	22.7126	0.000000
beta1	0.921341	0.003515	262.0871	0.000000

Robust Standard Errors:

	Estimate	Std. Error	t value	Pr(> t)
mu	0.007472	0.002059	3.6286	0.000285
ar1	-0.134144	0.083574	-1.6051	0.108474
ma1	0.203012	0.083442	2.4330	0.014976
omega	0.000254	0.000132	1.9237	0.054394
alpha1	0.077659	0.019266	4.0309	0.000056
beta1	0.921341	0.020438	45.0798	0.000000

LogLikelihood : -40868.21

Information Criteria

Information Criteria

Akaike	3.3885
Bayes	3.3905
Shibata	3.3885
Hannan-Quinn	3.3892

Weighted Ljung-Box Test on Standardized Residuals

	statistic	p-value
Lag[1]	3.028	0.081822
Lag[2*(p+q)+(p+q)-1][5]	5.785	0.000184
Lag[4*(p+q)+(p+q)-1][9]	11.552	0.001879

d.o.f=2

H0 : No serial correlation

Weighted Ljung-Box Test on Standardized Squared Residuals

	statistic	p-value
Lag[1]	25.85	3.697e-07
Lag[2*(p+q)+(p+q)-1][5]	26.59	2.868e-07
Lag[4*(p+q)+(p+q)-1][9]	30.02	4.309e-07
d.o.f=2		

Weighted ARCH LM Tests

	Statistic	Shape	Scale	P-Value
ARCH Lag[3]	0.05186	0.500	2.000	0.8199
ARCH Lag[5]	0.98237	1.440	1.667	0.7382
ARCH Lag[7]	4.72069	2.315	1.543	0.2543

Nyblom stability test

Nyblom stability test

Joint Statistic: 27.0206

Individual Statistics:

mu 0.3620

ar1 5.9721

ma1 6.0126

omega 0.2546

alpha1 8.2881

beta1 5.2529

Asymptotic Critical Values (10% 5% 1%)

Joint Statistic: 1.49 1.68 2.12

Individual Statistic: 0.35 0.47 0.75

Sign Bias Test

	t-value	prob	sig
Sign Bias	5.926	3.145e-09	***
Negative Sign Bias	1.155	2.481e-01	
Positive Sign Bias	1.020	3.075e-01	
Joint Effect	50.496	6.263e-11	***

Adjusted Pearson Goodness-of-Fit Test:

	group	statistic	p-value(g-1)
1	20	430.3	2.114e-79
2	30	505.8	1.884e-88
3	40	559.0	2.945e-93
4	50	615.8	4.854e-99

Elapsed time : 1.665252

Graphs for Bitcoin

Google Slides Link: [📄 UMich Prof: Bitcoin Analysis Graphs](#)